A Project Report on

VaultShield: Sandboxed, Monitored Execution and Human Review for AI Coding Agents

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

in

Computer Science and Engineering (Data Science)

by

Aayush Subhash Balip (24207010)
Krish Mahesh Patil (24207006)
Muhammad Anis Momin (24207008)
Sameer Hasan Saiyed (24207014)

Under the Guidance of

Prof. Rajashri Chaudhari
Prof. Hardiki Achrekar



Computer Science and Engineering (Data Science)
A.P. Shah Institute of Technology
G.B. Road, Kasarvadavali, Thane (W) – 400615
UNIVERSITY OF MUMBAI

Academic Year 2026–2027

Approval Sheet

This Project Report entitled ***VaultShield: Sandboxed, Monitored Execution and Human Review for AI Coding Agents*** submitted by **Aayush Subhash Balip** (24207010), **Krish Mahesh Patil** (24207006), **Muhammad Anis Momin** (24207008), and **Sameer Hasan Saiyed** (24207014) is approved for the partial fulfillment of the requirement for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering (Data Science)** from **University of Mumbai**.

Prof. Rajashri Chaudhari

Guide

Prof. Hardiki Achrekar

Co-Guide

Dr. Pravin Adivarekar

HOD, Computer Science and Engineering (Data Science)

Place: A.P. Shah Institute of Technology, Thane

Date:

CERTIFICATE

This is to certify that the project entitled ***VaultShield: Sandboxed, Monitored Execution and Human Review for AI Coding Agents*** submitted by **Aayush Subhash Balip** (24207010), **Krish Mahesh Patil** (24207006), **Muhammad Anis Momin** (24207008), and **Sameer Hasan Saiyed** (24207014) for the partial fulfillment of the requirement for award of a degree of **Bachelor of Engineering in Computer Science and Engineering (Data Science)** to the University of Mumbai, is a bonafide work carried out during academic year 2026–2027.

Prof. Rajashri Chaudhari	**Prof. Hardiki Achrekar**
Guide	Co-Guide
Dr. Pravin Adivarekar	**Dr. Uttam D. Kolekar**
HOD, C.S.E (Data Science)	Principal

External Examiner(s)	Internal Examiner(s)
1.	1.
2.	2.

Place: A.P. Shah Institute of Technology, Thane

Date:

Acknowledgement

We express our sincere gratitude to our project guide, **Prof. Rajashri Chaudhari**, for the constant guidance, encouragement and critical feedback provided throughout this project. The guide's insistence on precision, both in engineering and in stating the limits of what had been built, shaped the security model at the heart of this work.

We are equally grateful to our co-guide, **Prof. Hardiki Achrekar**, for valuable suggestions on system design and evaluation, and for the time spent reviewing our progress at every stage.

We thank **Dr. Pravin Adivarekar**, Head of the Department of Computer Science and Engineering (Data Science), for providing the facilities and the supportive environment that made this project possible, and **Dr. Uttam D. Kolekar**, Principal, A. P. Shah Institute of Technology, for the institutional support extended to us.

We also thank the project coordinators for organising the reviews and keeping us on schedule, and all the teaching and non-teaching staff of the department for their help throughout the academic year.

Finally, we thank our colleagues and friends for their feedback on the dashboard and the documentation, and our families for their patience and encouragement.

Aayush Subhash Balip
(24207010)

Krish Mahesh Patil
(24207006)

Muhammad Anis Momin
(24207008)

Sameer Hasan Saiyed
(24207014)

Declaration

We declare that this written submission represents our ideas in our own words, and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented, fabricated or falsified any idea, data, fact or source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
Aayush Subhash Balip (24207010)

(Signature)
Krish Mahesh Patil (24207006)

(Signature)
Muhammad Anis Momin (24207008)

(Signature)
Sameer Hasan Saiyed (24207014)

Date:

Abstract

AI coding agents now read repositories, edit many files and execute shell commands with little human intervention. Run directly on a developer's machine, such an agent inherits the developer's privileges: it can read real secrets, write anywhere, contact any host and leave unreviewed changes in version control. Current tooling conflates running an agent with trusting its output. This project presents VaultShield, a pipeline that separates the two. VaultShield runs the agent in a disposable Docker container holding a fresh clone of the repository and a read-only credential file whose values mirror the shape of the developer's .env but contain no real secret; a repository-host token, when needed, is confined to the single git clone process. Terminal output, in-container file changes and policy violations are merged into a timestamped JSON Lines event log. Writes to off-limits paths fail the run, and an optional egress policy routes traffic through a hardened proxy that blocks and records non-allowlisted hosts. A separate hardened helper container then rebuilds the repository from a pre-agent base and commits the change to a dedicated branch with a structured diff, so the agent cannot falsify the evidence. Artifacts are uploaded to a Django REST Framework backend on MongoDB, scanned asynchronously, and approved or rejected by a reviewer in a React dashboard. Only an approved run can become a pull or merge request. The system comprises about 18,100 logical lines of product code. Of 2,704 automated tests, including Docker-backed isolation, egress, token-hygiene and tamper-resistance tests and concurrency tests against a real MongoDB replica set, 2,702 passed. In a 40-run benchmark, VaultShield added 1.17 seconds of fixed overhead per run by default and 1.98 seconds with egress allowlisting, a projected 0.4–0.7% of a five-minute agent session; the host checkout was untouched and the change was captured in every run, with no container or network left behind. The evaluation also exposed limits: the shared-kernel boundary of containers, opt-in egress control, a polling file watcher that missed 20% of very short writes, and an approval gate that is a workflow control rather than a separation-of-duties control in the single-user MVP. VaultShield shows that containment, evidence and human review for coding agents can be delivered together at negligible cost, with every residual risk stated explicitly.

Keywords: AI coding agents, sandboxing, Docker, human-in-the-loop review, audit logging, credential isolation, egress control, software supply-chain security.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Objectives . . . 2
- 1.4 Scope . . . 3

2 Literature Review **4**
- 2.1 Comparative Analysis of Recent Studies . . . 6
- 2.2 Research Gap . . . 7

3 Project Design **8**
- 3.1 Existing System . . . 8
- 3.2 Proposed System . . . 9
- 3.3 Critical Components of System Architecture . . . 10
  - 3.3.1 Sandbox and Container Manager . . . 10
  - 3.3.2 Run Monitoring and Logging . . . 10
  - 3.3.3 Diff Capture on a Dedicated Branch . . . 10
  - 3.3.4 Review and Human Approval Service . . . 12
  - 3.3.5 Backend API and Dashboard . . . 13
  - 3.3.6 Supporting Module: Protect . . . 13
- 3.4 System Diagrams . . . 15
  - 3.4.1 UML Diagram . . . 15
  - 3.4.2 Activity Diagram . . . 17
  - 3.4.3 Use Case Diagram . . . 18
  - 3.4.4 Sequence Diagram . . . 19

4 Project Planning and Cost Estimation **20**
- 4.1 Project Development Methodology . . . 20
- 4.2 Project Schedule and Gantt Chart . . . 20
- 4.3 Team Roles and Responsibilities . . . 21
- 4.4 Software Size Estimation . . . 22
  - 4.4.1 KLOC-Based Estimation . . . 22
- 4.5 Effort Estimation Using COCOMO . . . 22
  - 4.5.1 Basic COCOMO Model . . . 22
  - 4.5.2 Calculation . . . 23
- 4.6 Human Resource Cost . . . 23
- 4.7 Hardware, Software and Cloud Infrastructure Cost . . . 23
- 4.8 Testing, Deployment and Maintenance Cost . . . 24
- 4.9 Total Estimated Project Cost . . . 24
- 4.10 Assumptions and Cost Justification . . . 24
- 4.11 Risk Assessment and Mitigation . . . 25

5 Project Implementation **26**
- 5.1 Code Snippets . . . 26

5.1.1 Launching the Sandbox Container . . . 26
5.1.2 One-Shot Credential for Private Clones . . . 27
5.1.3 Event Logging and Off-Limits Enforcement . . . 28
5.1.4 Egress Allow/Deny Decision . . . 29
5.1.5 Branch Commit and Structured Diff . . . 30
5.1.6 Approval State Machine . . . 31
5.2 Steps to Access the System . . . 32
5.3 System Implementation . . . 33
5.3.1 Command-Line Interface . . . 33
5.3.2 Landing Page and Projects Dashboard . . . 33
5.3.3 Run History . . . 34
5.3.4 Run Detail Views . . . 35
5.3.5 Approval Decision and Submission . . . 37
5.3.6 Implementation Summary . . . 37
5.4 Timeline Sem VII . . . 38

6 Testing **40**
6.1 Software Testing . . . 40
6.2 Types of Testing . . . 40
6.2.1 Functional Testing . . . 40
6.2.2 System Testing . . . 41
6.3 Testing Results and Findings . . . 41
6.4 Sample Test Cases . . . 42
6.5 Conclusion on Testing . . . 42

7 Result and Discussions **43**
7.1 Introduction . . . 43
7.2 Evaluation Metrics . . . 43
7.3 Performance Analysis . . . 43
7.4 Overhead with and without VaultShield . . . 45
7.5 Safety Events Captured . . . 46
7.6 Secret Exposure With and Without Protection . . . 47
7.7 Case Study: Analysis of Real Runs . . . 47
7.8 Overall Discussion and Limitations . . . 48
7.9 Chapter Summary . . . 48

8 Conclusion **49**

9 Future Scope **50**

Bibliography **51**

List of Figures

3.1 Existing workflow – direct agent execution on the developer's host 8
3.2 Proposed VaultShield pipeline 11
3.3 Per-run egress network topology 11
3.4 Run status state machine 12
3.5 Overall system architecture 14
3.6 UML class diagram of the core backend entities and CLI classes 16
3.7 Activity diagram of a VaultShield run, from agent kickoff to PR creation 17
3.8 Use case diagram 18
3.9 Sequence diagram of one complete approved run 19

5.1 Sandbox container creation and teardown 27
5.2 Authenticated clone with a one-shot credential helper 28
5.3 Event records, off-limits matching and the JSON Lines writer 29
5.4 Egress decision for one request target 30
5.5 Committing the run branch and computing the diff 31
5.6 Transactional approve/reject decision 32
5.7 Terminal output of a successful vaultshield run with the mock agent . . . 33
5.8 Landing page 34
5.9 Projects dashboard 34
5.10 Run history with status filters 35
5.11 Run detail – Diff tab 35
5.12 Run detail – Timeline tab 36
5.13 Run detail – Screenshots tab 36
5.14 Run detail – Security tab 37
5.15 Run detail after approval, showing the submit command 37
5.16 Gantt chart of the VaultShield major project 39

7.1 Mean stage latency in the default configuration 44
7.2 Mean end-to-end time by configuration 45

List of Tables

2.1 Comparative analysis of the reviewed literature 6

3.1 Comparison of the existing system and the proposed system 9

4.1 Planned project schedule for VaultShield 21
4.2 Team roles and responsibilities 22
4.3 Measured software size by component 22
4.4 Basic COCOMO estimate 23
4.5 Actual human-resource cash expenditure 23
4.6 Actual hardware, software and infrastructure cash expenditure 24
4.7 Actual testing, deployment and maintenance cash expenditure 24
4.8 Total actual project cash expenditure 24
4.9 Risk assessment and mitigation 25

6.1 Types of testing performed 40
6.2 Automated test suite results 41
6.4 Sample test cases 41

7.1 Evaluation metrics 43
7.2 Stage latency in the default configuration 44
7.3 End-to-end latency by configuration 44
7.4 Running an agent with and without VaultShield 45
7.5 Safety events and evidence captured 46
7.6 Secret exposure under four protection configurations 47
7.7 Residual risks and limitations 48

8.1 Achievement of objectives 49

List of Abbreviations

AA	:	Level AA conformance of the Web Content Accessibility Guidelines
ACI	:	Agent-Computer Interface
API	:	Application Programming Interface
ASGI	:	Asynchronous Server Gateway Interface
CI/CD	:	Continuous Integration / Continuous Delivery
CLI	:	Command-Line Interface
COCOMO	:	Constructive Cost Model
CORS	:	Cross-Origin Resource Sharing
CPU	:	Central Processing Unit
CWE	:	Common Weakness Enumeration
DNS	:	Domain Name System
DRF	:	Django REST Framework
HTTP	:	Hypertext Transfer Protocol
HTTPS	:	Hypertext Transfer Protocol Secure
IDE	:	Integrated Development Environment
IP	:	Internet Protocol
JSON	:	JavaScript Object Notation
JSONL	:	JSON Lines
KLOC	:	Thousand Lines of Code
LLM	:	Large Language Model
LM	:	Language Model
MR	:	Merge Request
MVP	:	Minimum Viable Product
OAuth	:	Open Authorization
OCR	:	Optical Character Recognition
PM	:	Person-Month
PR	:	Pull Request
PTY	:	Pseudo-Terminal
RBAC	:	Role-Based Access Control
REST	:	Representational State Transfer

SHA	:	Secure Hash Algorithm
SLOC	:	Source Lines of Code
SNI	:	Server Name Indication
SPA	:	Single-Page Application
SQL	:	Structured Query Language
STRIDE	:	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS	:	Transport Layer Security
TTL	:	Time To Live
UI	:	User Interface
UML	:	Unified Modeling Language
URL	:	Uniform Resource Locator
UTC	:	Coordinated Universal Time
VCS	:	Version Control System
VM	:	Virtual Machine
WCAG	:	Web Content Accessibility Guidelines
WSGI	:	Web Server Gateway Interface

Chapter 1

Introduction

Large Language Models trained on source code have moved from completing a single line to operating as autonomous coding agents that read a repository, plan a change, edit many files, run shell commands and iterate on test results with little human intervention. Tools such as OpenAI Codex CLI, Google Antigravity, Amazon's Kiro and Anthropic's Claude Code expose this capability directly in the developer's terminal or editor, and research systems such as SWE-agent [3] and OpenHands [4] resolve a meaningful fraction of real GitHub issues from benchmarks like SWE-bench [2].

The same capability makes coding agents hazardous. An agent is an ordinary process that runs with the privileges of the user who launched it. On a developer's workstation it can read every secret in the shell environment and in nearby .env files, write anywhere the user can write, run commands the prompt never asked for, contact arbitrary hosts and leave changes in version control before anyone has examined them. Prior work shows that LLM-generated code is often insecure [8], that LLM-integrated applications can be hijacked through indirect prompt injection [7], and that even carefully aligned agents take risky actions in a non-trivial fraction of high-stakes scenarios [5]. Governance guidance recommends constraining an agent's action space, requiring human approval for consequential actions and making agent activity legible [9], [10], but these recommendations are rarely packaged as a pipeline a developer can run against an existing repository.

VaultShield addresses that gap. It runs an AI coding agent inside a disposable Docker container, records what the agent does, captures the resulting code changes as a reviewable Git diff on a dedicated branch, and holds those changes behind an explicit human approval step before they can become a pull request or merge request. The developer's working tree, real credentials and repository write access are never handed to the agent. The system consists of a Python command-line interface that orchestrates the sandboxed run, a per-run Docker execution plane, and a Django REST Framework backend with a React dashboard where a reviewer approves or rejects the run. Producing evidence therefore never implies accepting it.

1.1 Motivation

The first motivation is the asymmetry between the cost of an agent mistake and the cost of preventing it. An agent that deletes the wrong directory or commits a hard-coded key can cost hours of recovery, or far more if credentials leak, while running the same agent in a throwaway container costs about 1.2 seconds of fixed overhead per run on the evaluation machine.

The second motivation is credential exposure. Applications keep database strings, API keys and session secrets in a local .env file inside the very repository the agent is asked to modify. A preliminary spike in this project confirmed the risk: with no protection, a production coding agent asked to reveal a value from .env read the file with its built-in tool and printed both test secrets. Agent-specific deny rules reduced but did not remove the exposure, and only keeping the real value off disk proved reliable.

The third motivation is reviewability. Code review is the primary quality-control mechanism of modern software engineering [14], yet agent output often bypasses the pull-request workflow because it lands directly in the working tree. Capturing agent output on a dedicated branch, together with a timestamped event log and screenshots, lets a reviewer apply the same review process to agent changes and leaves a durable audit trail.

The fourth motivation is honest security engineering. Container isolation shares the host kernel and is not equivalent to a virtual machine boundary [11], [12], [13]. VaultShield therefore states which controls it implements and which risks remain, so that adopters can judge where it fits in their own threat model.

1.2 Problem Statement

Given a Git repository, a natural-language prompt and an AI coding agent, the problem is to let the agent attempt the requested change while guaranteeing that it never executes against the developer's real working tree, real application secrets or repository-host credentials; that everything it does is recorded as reviewable evidence; that its change is captured in a form it could not have tampered with; and that no change reaches a shared branch or pull request without an explicit human approval decision.

The agent must still be able to work, so it needs a real repository, a functioning toolchain and sometimes its own model API credential. The application must be able to start inside the sandbox for screenshots, so configuration values must keep their structural shape even though their content is fake. Every container, network and temporary branch must be removed on every exit path, and the pipeline overhead must stay small compared with typical agent run times.

1.3 Objectives

The project set out to achieve the following objectives.

1. Develop a universal AI agent execution platform compatible with multiple AI coding agents and development environments through a common CLI interface.
2. Implement an intelligent Agentic AI Orchestrator that understands project context, asks clarification questions before execution, and provides enriched context to AI agents to improve accuracy and reduce hallucinations.
3. Ensure secure AI-assisted software development by executing AI agents inside isolated Docker-based sandbox environments while protecting production credentials and resources.
4. Provide comprehensive monitoring and automated security analysis by capturing AI activities, execution logs, code changes, and identifying security vulnerabilities or unsafe operations.
5. Enable transparent governance and human approval through a web-based dashboard that allows developers to review, verify, approve, or reject AI-generated changes before deployment.

1.4 Scope

The scope of the VaultShield project covers the sandboxed execution, monitoring and human review of AI coding agents working on a Git repository.

- **Command-Line Interface:** the vaultshield CLI with the init, login, run, submit and teardown commands and the supporting protect command group.
- **Sandboxed Execution:** a per-run Docker sandbox with a fresh clone, a read-only dummy credential file and guaranteed teardown.
- **Agent Adapters:** the mock, codex, kiro and antigravity agents behind a common adapter interface.
- **Monitoring and Egress Control:** terminal capture, in-container file watching, off-limits paths, a JSON Lines event log and an optional egress policy enforced by a hardened proxy.
- **Trusted Change Capture:** the run branch and structured diff built in a separate hardened helper container, with run outputs scanned for leaked credentials.
- **Backend, Dashboard and Submission:** a Django REST Framework backend on MongoDB with the run state machine and Celery-based scanning, a React review dashboard, and pull or merge request submission to GitHub or GitLab for approved runs.

The following are outside the scope of the project:

- **Stronger Isolation and Hosted Execution:** virtual-machine-grade isolation and running the CLI as a hosted service; container escape is a documented residual risk.
- **Behavioural Judgement:** automatic analysis of agent intent, content-level network filtering and detection of deliberately encoded exfiltration.
- **Multi-User Operation:** multi-tenant operation, role-based access control and separate runner and reviewer identities.
- **Other Integrations:** live output streaming, other repository hosts, automatic merging and CI/CD pipeline integration.

The minimum viable product is scoped to a single developer's machine, so the approval gate is a workflow control within one trust domain rather than a security boundary between mutually distrustful parties.

Chapter 2

Literature Review

The literature relevant to VaultShield falls into five overlapping strands: the capabilities and evaluation of autonomous coding agents, the security risks of LLM agents and of the code they generate, the governance and visibility of agentic systems, isolation technologies for running untrusted code, and human code review. Each strand is reviewed below, followed by a comparative analysis and a statement of the research gap that the project addresses.

Code-generating models and the need for sandboxes. Chen et al. [1] introduced Codex, a GPT model fine-tuned on public GitHub code, together with the HumanEval benchmark of hand-written programming problems scored by functional correctness. Because scoring functional correctness requires executing model-generated programs, the authors ran those programs inside a gVisor-based sandbox and discussed misuse, over-reliance and insecure code as broader risks. The paper established, at the start of the code-LLM era, that model output must be treated as untrusted code and executed only behind an isolation boundary. Its sandbox, however, was an internal evaluation harness for short, self-contained functions rather than a developer-facing tool for repository-scale changes with human review.

Repository-scale agents. Jimenez et al. [2] proposed SWE-bench, 2,294 task instances drawn from real GitHub issues and pull requests across twelve popular Python repositories, in which a model must produce a patch that makes the repository's failing tests pass. SWE-bench moved evaluation from isolated functions to whole repositories, and the best model of the time resolved fewer than 2% of the issues. Yang et al. [3] then showed with SWE-agent that a purpose-built agent-computer interface, with dedicated commands for viewing, searching and editing files, raises this to 12.5% with GPT-4 Turbo, executing inside a Docker container. Wang et al. [4] presented OpenHands, an open platform in which agents act through a sandboxed Docker runtime and every action and observation is recorded in an event stream. These systems validate two of VaultShield's design choices: a container is the practical execution environment for repository-scale agents, and a chronological record of agent actions is the natural unit of observability. Their sandboxes, however, exist to give the agent a reproducible workspace. None of them holds the agent's patch for human approval, and none keeps the developer's real secrets out of the workspace.

Risks of LLM agents. Ruan et al. [5] introduced ToolEmu, which uses a language model to emulate tool execution so that agents can be red-teamed against high-stakes scenarios without real side effects, together with an automatic safety evaluator. Across 36 toolkits and 144 test cases, even the safest agent they evaluated showed risky failures in 23.9% of cases. Debenedetti et al. [6] built AgentDojo, a dynamic environment with 97 realistic tasks and 629 security test cases for measuring prompt-injection attacks against tool-using agents and the defences against them, and found that both remain far from solved. Greshake et al. [7] demonstrated indirect prompt injection, in which instructions planted in data that an LLM-integrated application retrieves take control of the application and can cause data exfiltration. For coding agents the implication is direct, because a repository's own files, issues and dependencies are untrusted input.

These works characterise the threat convincingly, but they are evaluation frameworks and do not provide an operational containment and review pipeline for developers.

Security of AI-generated code. Pearce et al. [8] prompted GitHub Copilot with 89 scenarios based on MITRE's top Common Weakness Enumeration categories and found that about 40% of the 1,689 generated programs contained vulnerabilities. This result motivates VaultShield's static security scan over every ingested diff and, more fundamentally, its requirement that a human reviews agent code before it is merged. The study evaluated completion suggestions rather than autonomous agents, so it does not cover agents that also execute commands; VaultShield extends the concern from insecure code to insecure behaviour.

Governance and visibility of agentic systems. Shavit et al. [9] proposed practices for keeping agentic AI systems safe and accountable, including evaluating suitability for a task, constraining the action space, requiring human approval for high-impact actions, setting safe defaults, making agent activity legible, automatic monitoring, attributability and interruptibility. Chan et al. [10] argued that oversight of AI agents depends on visibility and identified three categories of measures: agent identifiers, real-time monitoring and activity logging. VaultShield can be read as a concrete implementation of several of these practices for coding agents. The run identifier names the sandbox, the branch, the run directory and the backend record; the event log, egress log and screenshots form the activity log; and the approval state machine is the human-approval control. Both papers are deliberately technology-neutral, so they leave the engineering of such controls as an open problem.

Container isolation and its limits. Combe et al. [11] analysed Docker from a security perspective and identified the main areas of concern: isolation through Linux namespaces and control groups, the shared host kernel, network configuration and the trustworthiness of the image distribution chain. They concluded that Docker's default configuration offers reasonable isolation but that its security depends heavily on configuration. Agache et al. [12] described Firecracker, the virtual machine monitor behind AWS Lambda, whose microVMs boot in as little as 125 ms with under 5 MiB of memory overhead, showing that VM-grade isolation can approach container-like efficiency. Young et al. [13] measured gVisor, a user-space kernel that intercepts system calls to shrink the host kernel's attack surface, and showed that this stronger isolation carries substantial costs for system-call-heavy and file-system-heavy workloads. Together these papers frame VaultShield's trade-off: Docker containers are cheap and widely available but share the kernel, while gVisor and microVMs offer stronger boundaries at a cost in performance or operational complexity. VaultShield uses Docker for its MVP, documents the escape risk and lists gVisor and microVMs as future work.

Human code review. Bacchelli and Bird [14] studied modern code review at Microsoft through observation, interviews and surveys. Although finding defects is the main stated motivation for review, much of its actual value lies in knowledge transfer, team awareness and alternative solutions, and the main challenge for reviewers is understanding the change and its context. This finding shapes VaultShield's dashboard: the reviewer receives not only the diff but also the timeline of what the agent did, screenshots of the running application and automated findings, so that understanding an agent's change does not require reconstructing its reasoning from the patch alone.

2.1 Comparative Analysis of Recent Studies

Table 2.1 compares the reviewed works and records the limitation of each that VaultShield addresses.

Table 2.1: Comparative analysis of the reviewed literature

Ref.	Approach	Limitation relative to this project
[1] Chen et al., 2021	Code LLM evaluated by running generated programs in a sandbox	Internal harness for short functions; no repository workflow, credential model or review
[2] Jimenez et al., 2024	Repository-level benchmark from real issues	Measures capability; patches are applied for scoring, not held for approval
[3] Yang et al., 2024	Agent with a purpose-built agent-computer interface in Docker	Container is a workspace, not a trust boundary; no secret substitution, egress control or review gate
[4] Wang et al., 2025	Open agent platform with an event stream and sandboxed runtime	Changes are not captured on a reviewable branch behind an approval state machine
[5] Ruan et al., 2024	LM-emulated sandbox for red-teaming agents	Emulation cannot contain a real agent acting on a real repository
[6] Debenedetti et al., 2024	Benchmark for prompt-injection attacks and defences	Evaluates attacks; offers no containment or audit pipeline
[7] Greshake et al., 2023	Indirect prompt injection via retrieved data	Characterises the threat; mitigation is left open
[8] Pearce et al., 2022	Security study of AI code suggestions	Studies suggestions, not agents that also execute commands
[9] Shavit et al., 2023	Governance practices for agentic AI	Technology-neutral; no implementation for coding agents
[10] Chan et al., 2024	Visibility measures for AI agents	Defines what to log, not how to capture tamper-resistant evidence
[11] Combe et al., 2016	Security analysis of Docker	Does not address agent credentials or review
[12] Agache et al., 2020	Firecracker microVM isolation	Requires KVM; not integrated with a review workflow
[13] Young et al., 2019	Performance study of gVisor	Quantifies isolation cost; is not an agent pipeline
[14] Bacchelli & Bird, 2013	Empirical study of modern code review	Concerns human changes; agent changes need behavioural context

2.2 Research Gap

The literature shows that coding agents can now make repository-scale changes [2], [3], [4], that their code and actions are risky in measurable ways [5], [6], [7], [8], that container security depends on configuration [11], [12], [13], and that governance research recommends human approval and legible logs [9], [10]. Agent vendors have since shipped security features of their own: OS-level sandboxes for the commands an agent runs in Claude Code and the OpenAI Codex CLI, a firewalled hosted environment for the GitHub Copilot cloud agent, local microVM sandboxes in Docker Sandboxes, and a Docker runtime with an event stream in OpenHands. Table 2.2 compares them with VaultShield, based on their official documentation.

Table 2.2: Comparison with existing AI coding-agent security solutions

Control	Claude Code	Codex CLI	Copilot cloud agent	Docker Sandboxes	OpenHands	VaultShield
Agent compatibility	Claude Code only	Codex only	Copilot only	Several agents	Its own agents	Any agent with an adapter (four built)
Isolation approach	OS sandbox for shell commands	OS sandbox	Hosted ephemeral environment	Local microVM	Docker container	Docker container per run (shared kernel)
Real .env kept from agent	–	–	Only copilot-environment secrets	–	–	Shape-preserving dummy values
Host checkout protected	–	–	Yes (hosted)	–	Configurable	Fresh clone; host never mounted
Network restricted by default	Configurable	Yes	Yes (firewall)	Configurable	–	Opt-in allowlist proxy
Monitoring and audit evidence	–	–	Session logs	–	Event stream	Event log, egress log, screenshots
Change record outside agent's control	–	–	–	–	–	Separate capture helper
Human approval before repository submission	Per-action prompts	Per-action approval policy	Review after the branch is pushed	–	Per-action confirmation	Whole change held until approved

"–" means the cited documentation does not describe the control. The comparison exposes four gaps. First, each sandbox protects only its own vendor's agent, with no common policy or evidence format. Second, the project's .env sits inside the workspace that local sandboxes allow, and deny rules proved brittle in our spike (Section 7.6). Third, any activity record is produced beside the agent, which could tamper with Git state to misreport its change. Fourth, approval is per action or comes after the change is pushed. Existing products lead on isolation strength and default-deny networking. VaultShield closes the four gaps together for any agent on the developer's machine: dummy credentials and a one-shot clone token keep real secrets away, a separate capture helper builds the change record, and an approval gate holds the change until it is approved.

Chapter 3

Project Design

3.1 Existing System

Developers currently run AI coding agents in one of three ways. The most common is raw local execution, where the agent's CLI or editor extension runs in the developer's own checkout with the developer's environment, permissions and network access, and its changes appear as uncommitted edits. Some products add permission prompts or per-tool deny rules, but these are product-specific and often relaxed by users who find repeated prompts tedious. The second practice is ad-hoc containerisation, which usually bind-mounts the real working tree, so every write still lands on the host and the real .env file travels into the container. The third, development containers and cloud workspaces, is designed for convenience rather than for constraining an untrusted actor and holds nothing for review. Figure 3.1 summarises this workflow.

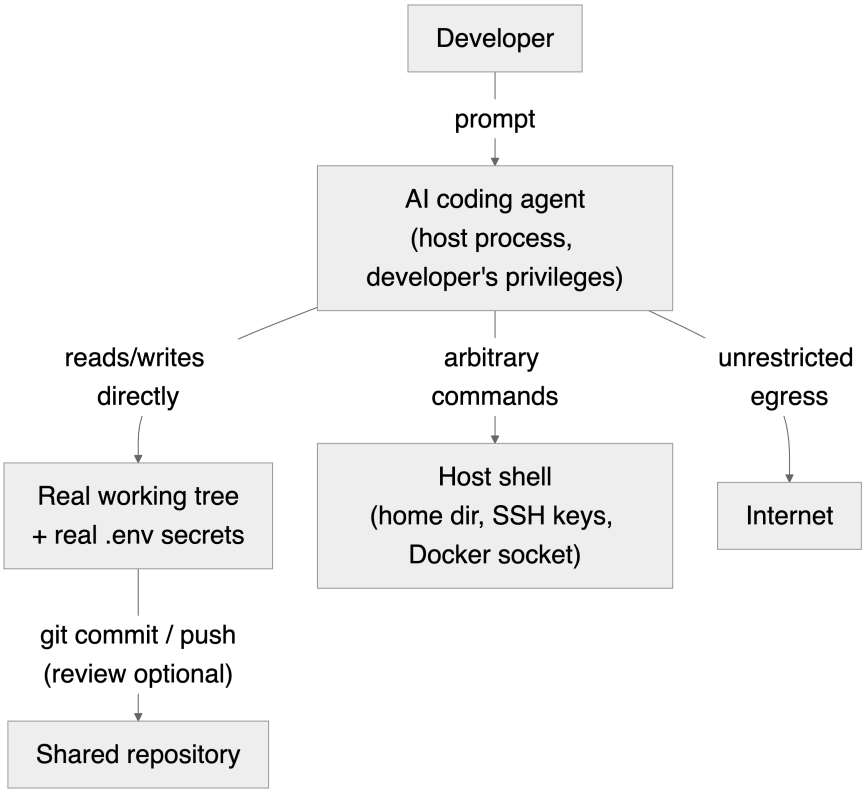


Figure 3.1: Existing workflow – direct agent execution on the developer's host

As Figure 3.1 shows, execution and acceptance are a single step and every arrow out of the agent touches a real asset. There is no isolation of side effects, no separation between the agent and the developer's secrets, no structured record of what happened, no enforced review before changes are committed, no automatic cleanup of processes the agent started,

and no record of which hosts the agent contacted.

3.2 Proposed System

VaultShield replaces this single step with a pipeline in which execution, evidence collection, storage and review are separate stages. The developer runs vaultshield run with an agent, a prompt and a repository URL. The CLI loads the project configuration, generates a run identifier, creates a per-run directory and registers the run with the backend before any container exists, so that later failures can be recorded. It then writes a dummy credential file that mirrors the shape of the developer's .env and resolves the agent adapter.

The CLI then creates a Docker container, mounts only the dummy credential file, clones the repository inside the container and installs dependencies. If egress control is enabled, the sandbox is moved onto an internal network whose only peer is a filtering proxy. Before any agent code runs, the CLI records the pre-agent commit through the Docker API as a trusted base. The agent runs under a pseudo-terminal while an in-container watcher records file changes, and both streams are merged into a timestamped event log; configured routes are screenshotted with a headless browser. The sandbox is then killed, and a separate hardened capture helper rebuilds the repository from the trusted base and a filtered copy of the work tree, commits the agent's changes to a dedicated run branch and computes the structured diff. Outputs are scanned for the agent's own credentials, the sandbox is torn down and the artifacts are uploaded, moving the run to awaiting_review. A reviewer inspects the evidence in the dashboard and approves or rejects the run, and only an approved run can be submitted as a pull or merge request. Figure 3.2 presents this pipeline.

Table 3.1 contrasts the proposed system with the existing practices.

Table 3.1: Comparison of the existing system and the proposed system

Concern	Existing practice	Proposed system
Working tree	Real checkout, modified in place	Fresh clone inside the container; host tree never mounted
Secrets	Real .env readable by the agent	Shape-preserving dummy values in a read-only .env.sandbox
Repository token	Often visible to the agent	Given only to the single git clone process
Network	Unrestricted, unlogged	Optional allowlist through a hardened proxy, every decision logged
Evidence	Terminal scroll-back	Event log, egress log, screenshots and structured diff
Change capture	Uncommitted edits	One commit on a run branch built by a separate helper
Review	Optional and informal	Enforced state machine; PR only from approved

Concern	Existing practice	Proposed system
Cleanup	Manual	Guaranteed teardown and a scheduled reaper

3.3 Critical Components of System Architecture

The architecture has three planes: a host-side CLI pipeline, a per-run Docker execution plane, and a backend and dashboard review plane. The critical components are described below.

3.3.1 Sandbox and Container Manager

The sandbox manager owns exactly one Docker container per run. It verifies that the credential file exists, creates a container named after the run, mounts the dummy credential file read-only, clones the repository inside the container and installs dependencies. The agent is executed through exec calls rather than as the container's main process, so the container's lifetime bounds every process the agent starts. For private repositories, the repository-host token is passed only as the environment of the single git clone process, so it never appears in the container's environment, the clone's Git configuration or any later process, and it is sent only to hosts on a strict per-provider allowlist. The Docker socket is never mounted.

When egress control is enabled, the EgressGuard builds the per-run topology shown in Figure 3.3: an internal network with no gateway, an ordinary bridge, and a hardened proxy attached to both. The sandbox is moved onto the internal network before the agent runs, so its only reachable peer is the proxy, which checks every destination host against the allowlist and logs each decision.

Any error during creation removes the partial container and raises a single SandboxError whose message never contains a token. Teardown force-removes the container and is called from a finally block, so it runs on success, failure, timeout and interruption alike.

3.3.2 Run Monitoring and Logging

The monitor produces the run's event log, events.jsonl, in which each line records an event type, a timestamp and a payload. The three event types are terminal output from the agent's pseudo-terminal, file changes observed by an in-container watcher, and violations such as off-limits writes, egress denials and leaked credentials. Each changed path is checked against the project's off-limits paths, and a match fails the run only after the log has been written, so evidence always outlives the failure. Browser capture starts the application inside the sandbox and takes one screenshot per configured route. Token hygiene scans the diff, the event log and changed files for the agent's own credential bytes; a hit fails the run, quarantines the diff and scrubs uploaded payloads.

3.3.3 Diff Capture on a Dedicated Branch

If the branch and diff were built inside the agent's own sandbox, a hostile agent could tamper with Git state to misrepresent its changes. VaultShield therefore reads the pre-agent commit through the Docker API before the agent runs, and after the agent finishes

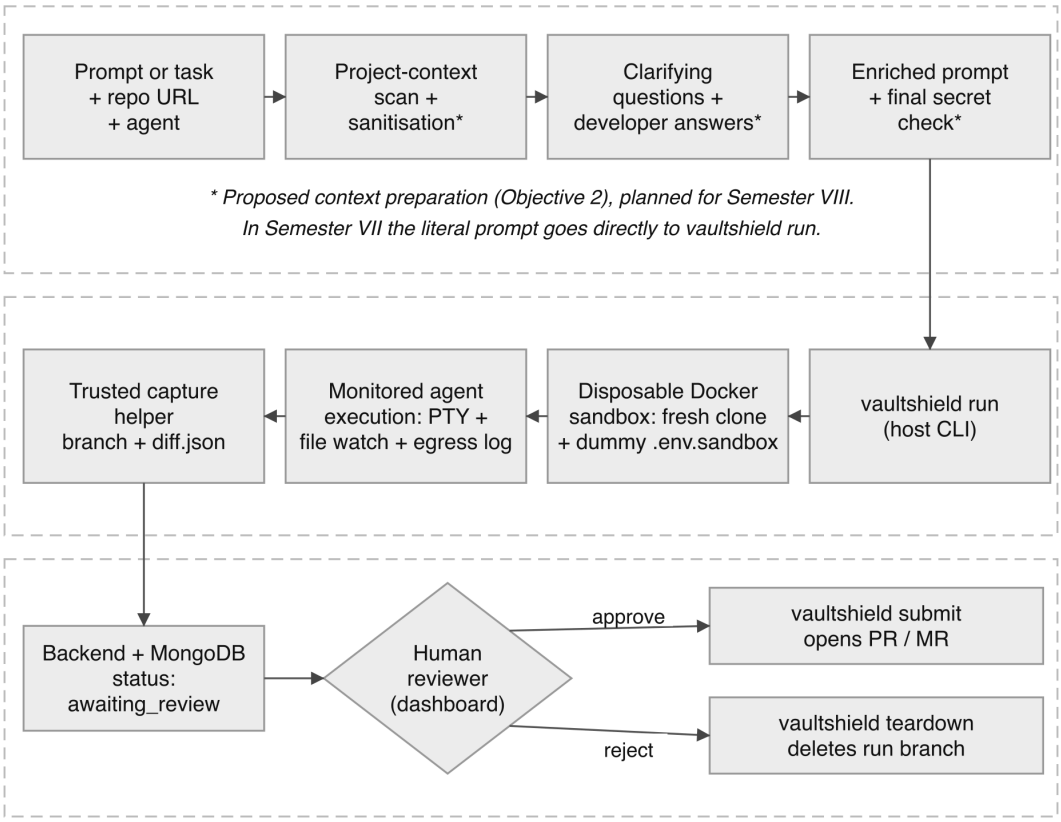


Figure 3.2: Proposed VaultShield pipeline

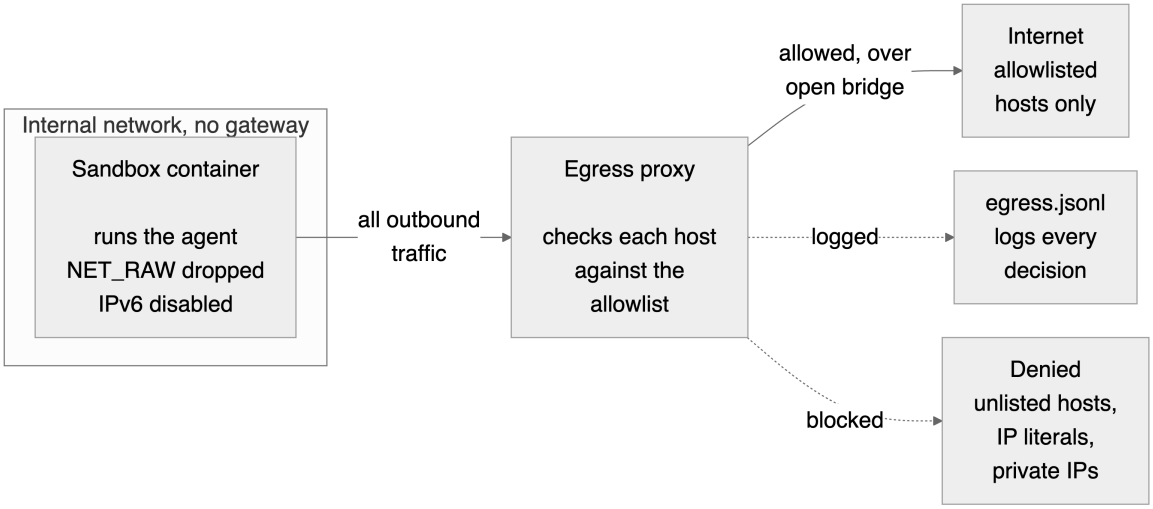


Figure 3.3: Per-run egress network topology

it kills the sandbox and extracts a filtered archive of the work tree. A fresh capture helper container with no network, no capabilities and no environment rebuilds the repository from the trusted base, refuses unsafe Git configuration, commits every change to the run branch as exactly one commit, and writes diff.json with per-file additions, deletions and size-capped patches. The branch is exported as a Git bundle because the helper is destroyed at the end of the run.

3.3.4 Review and Human Approval Service

The Run.status field is a state machine, shown in Figure 3.4. The CLI may move a run from running to awaiting_review or failed, and the approve and reject endpoints change the status only if it is still awaiting_review, inside a MongoDB transaction that also creates the Approval record. Of two concurrent decisions exactly one succeeds, and the other receives HTTP 409 with the run's actual status.

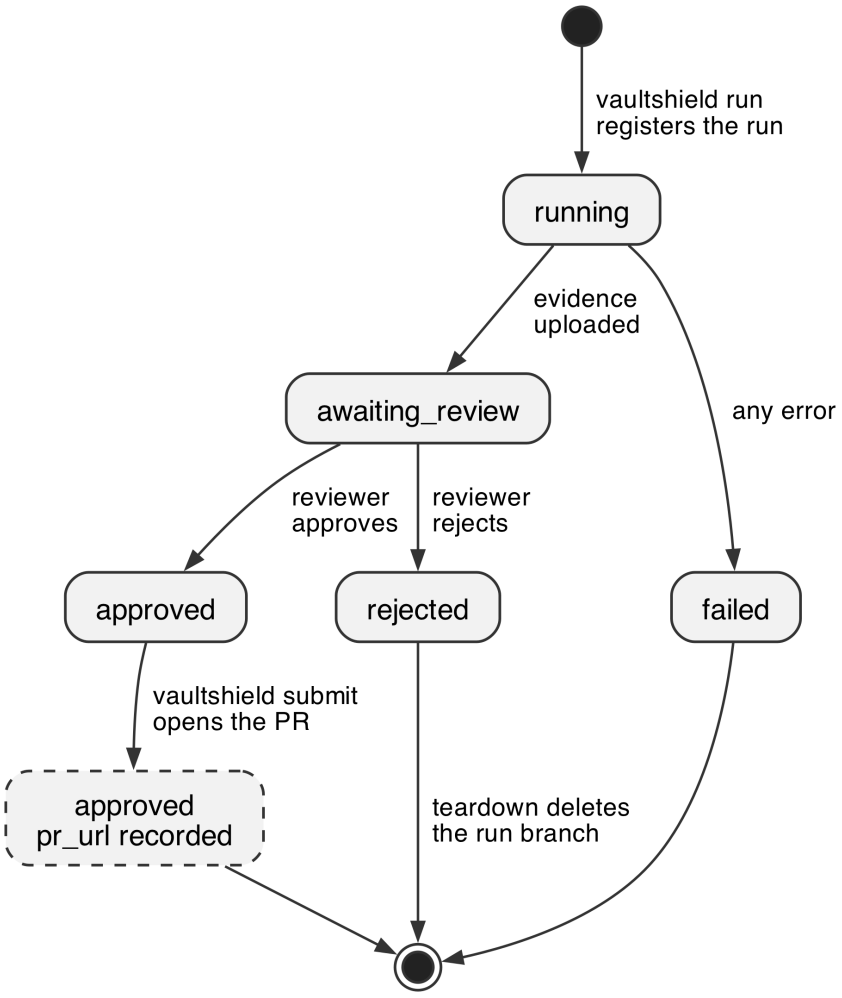


Figure 3.4: Run status state machine

vaultshield submit re-reads the run and refuses before any side effect unless its status is approved. For an approved run it checks that the current checkout matches the original repository, imports the branch from the verified bundle, pushes it without force, opens the pull or merge request and records its URL on the run. vaultshield teardown deletes the branch of a rejected run.

3.3.5 Backend API and Dashboard

The backend is a Django 5.2 project with Django REST Framework and MongoDB as its system of record. Every API path except login requires token authentication. The CLI registers runs, uploads events, diffs and screenshots, and patches the status; the dashboard reads run details and records decisions. Celery workers with Redis run a static security scan over the added lines of each diff, reap orphaned sandboxes every 15 minutes and flag approved but unsubmitted runs. The dashboard is a React 19 application built with Vite 7 that shows a projects overview, a filterable run history and a run detail page with Diff, Timeline, Screenshots and Security tabs and the decision controls.

3.3.6 Supporting Module: Protect

Protect covers agents that developers still run outside the sandbox: it moves every .env value into the macOS Keychain, leaves a placeholder in the file and injects the real values only into the application started through vaultshield exec.

Figure 3.5 shows all components and the data flows between them. The CLI is the only component that talks to the Docker daemon, the dummy credential file is the only host path mounted into the sandbox, artifacts reach the backend over authenticated HTTP, and the submit command is the only component that pushes to the repository host, after confirming approval with the backend.

3.3.7 Proposed Agentic AI Orchestration

The Agentic AI Orchestrator of Objective 2 is planned for Semester VIII and has not been implemented in Semester VII; in the evaluated system the developer passes a literal prompt to vaultshield run. Its purpose is to replace a vague one-line prompt, which forces an agent to guess the framework, the files to change and the constraints, with a structured prompt, so that fewer changes are rejected at review and fewer files, APIs or packages are hallucinated. As the first row of Figure 3.2 and the context orchestrator in Figure 3.5 show, it will run on the host before any sandbox is created. A context scanner will collect a bounded, non-secret project summary: the directory listing to three levels, the start command and port detected by vaultshield init, dependency names, the start of the README and recent commit subjects. A sanitiser will replace protected values and secret-shaped strings with *** and block the request if anything secret-like remains; .env and key files are never opened. A language model will then write three to five clarifying questions in a strict JSON format, the developer will answer them at the terminal or from a file, and a prompt builder will combine the task, the project summary, the answers, a fixed Constraints section that the model cannot change, and a suggested approach. A final secret check will run before the prompt is saved as prompt.md in the run directory and passed unchanged into the existing sandboxed pipeline, where the reviewer sees it beside the diff and timeline. Because the Orchestrator sends project context to an external model and could be steered by hostile README text, it will gain no privileges: the agent still receives only dummy credentials, its change still passes trusted capture and human approval, and pattern-based sanitisation remains a stated residual risk. Its expected benefit, higher task success and approval rates with fewer hallucinated changes, will be measured on real agents in Semester VIII.

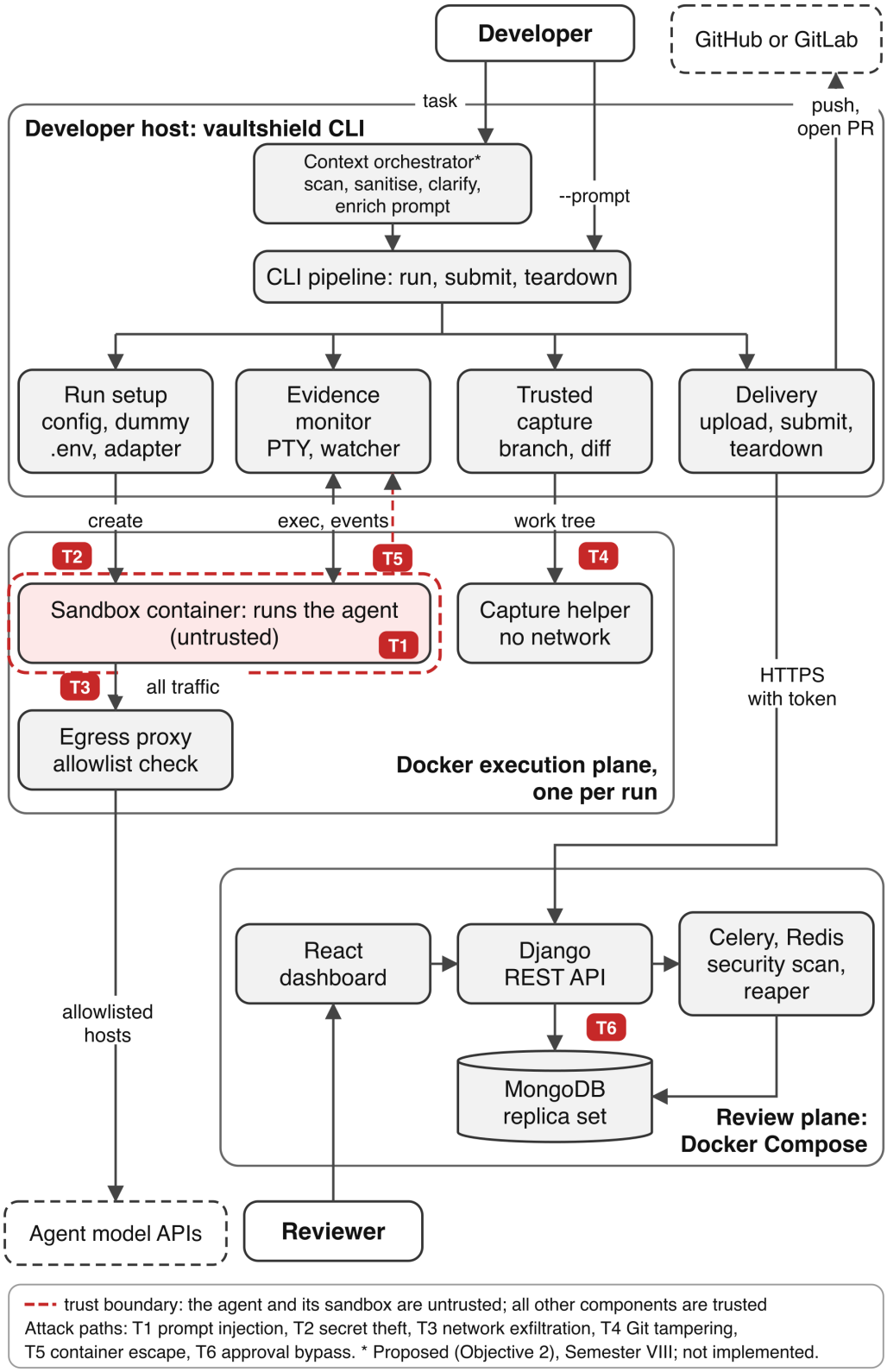


Figure 3.5: Overall system architecture

3.4 System Diagrams

System diagrams give a visual representation of the structure, behaviour and interactions of the proposed VaultShield system. The diagrams used are the UML Diagram, Activity Diagram, Use Case Diagram and Sequence Diagram.

3.4.1 UML Diagram

Figure 3.6 shows the persistent backend entities and the principal CLI-side classes. Run is the hub of the schema: RunEvent, Screenshot and SecurityFinding are many-per-run, while RunDiff and Approval are one-to-one, which makes "a run has exactly one change set and at most one decision" a schema-level guarantee. No backend field can store a repository-host token. On the CLI side, an AgentAdapter describes how its agent runs inside the Sandbox, the EgressGuard isolates the Sandbox's network, monitoring produces the RunEvent records, and the TrustedCapture helper produces the file entries stored in RunDiff.

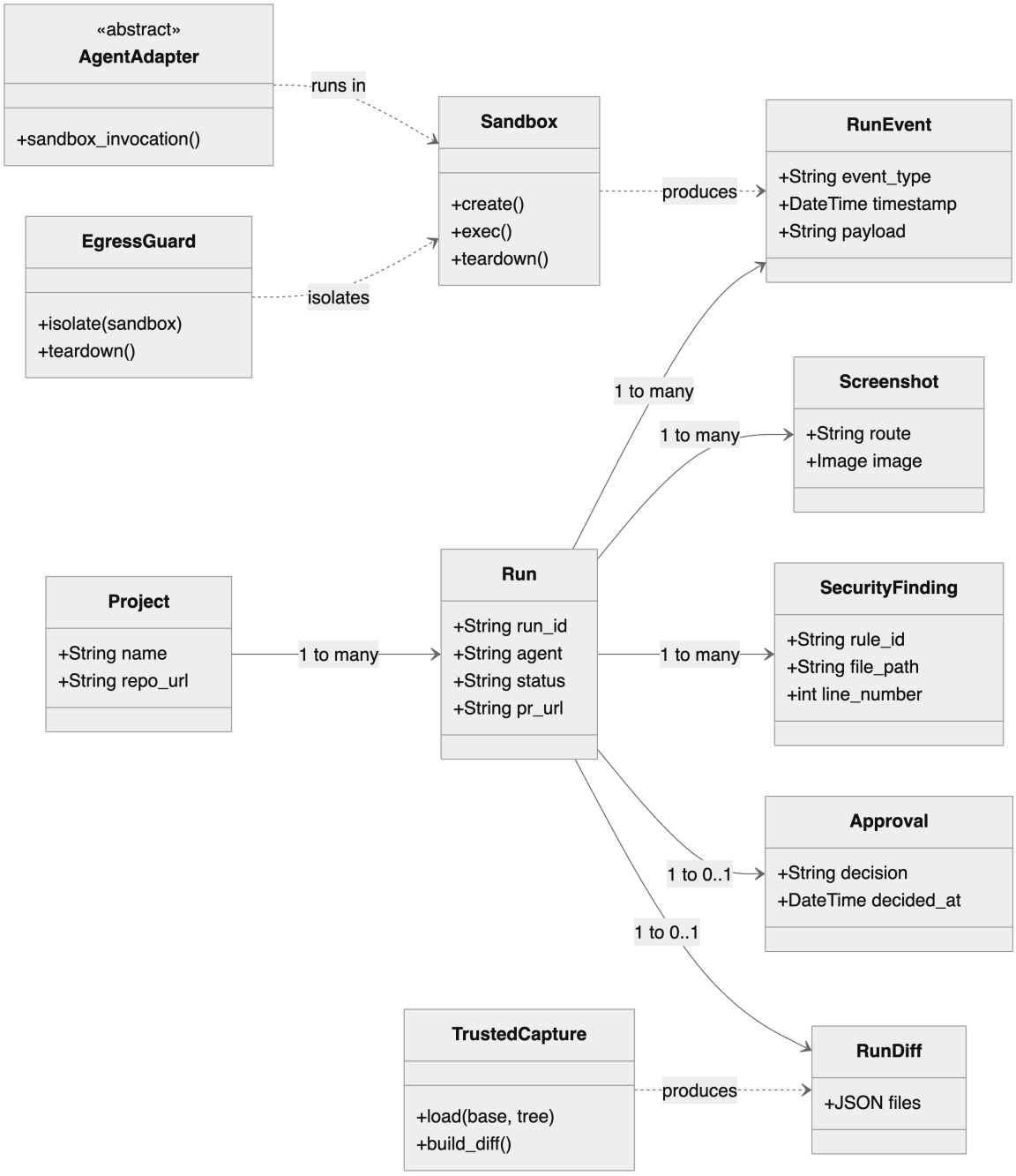


Figure 3.6: UML class diagram of the core backend entities and CLI classes

3.4.2 Activity Diagram

Figure 3.7 traces a run from setup through review and submission, reading each row in the direction of its arrows. The run is registered before the sandbox is created, policy failures are recorded, teardown runs on every exit path, and reviewer approval is the only route to submission.

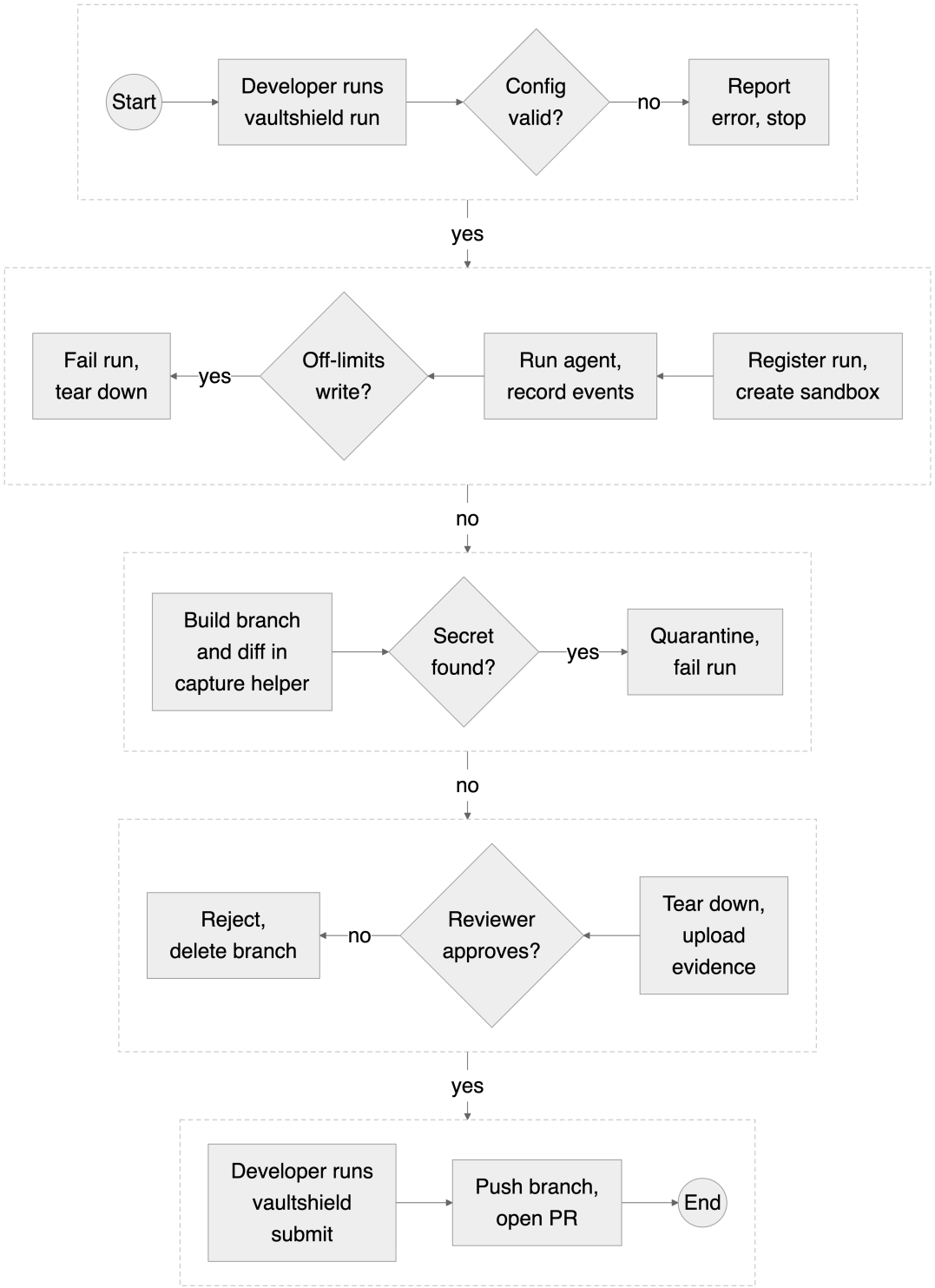


Figure 3.7: Activity diagram of a VaultShield run, from agent kickoff to PR creation

3.4.3 Use Case Diagram

Figure 3.8 shows the VaultShield actors and their interactions. The Developer configures and executes sandboxed runs, then submits or tears them down. The Reviewer inspects run evidence and approves or rejects the run. The System actor represents scanning and cleanup by Celery workers, and the Repository host receives the pushed branch and pull request. Executing a run includes dummy-credential generation, monitoring, change capture and upload, and may be extended by egress enforcement and screenshot capture.

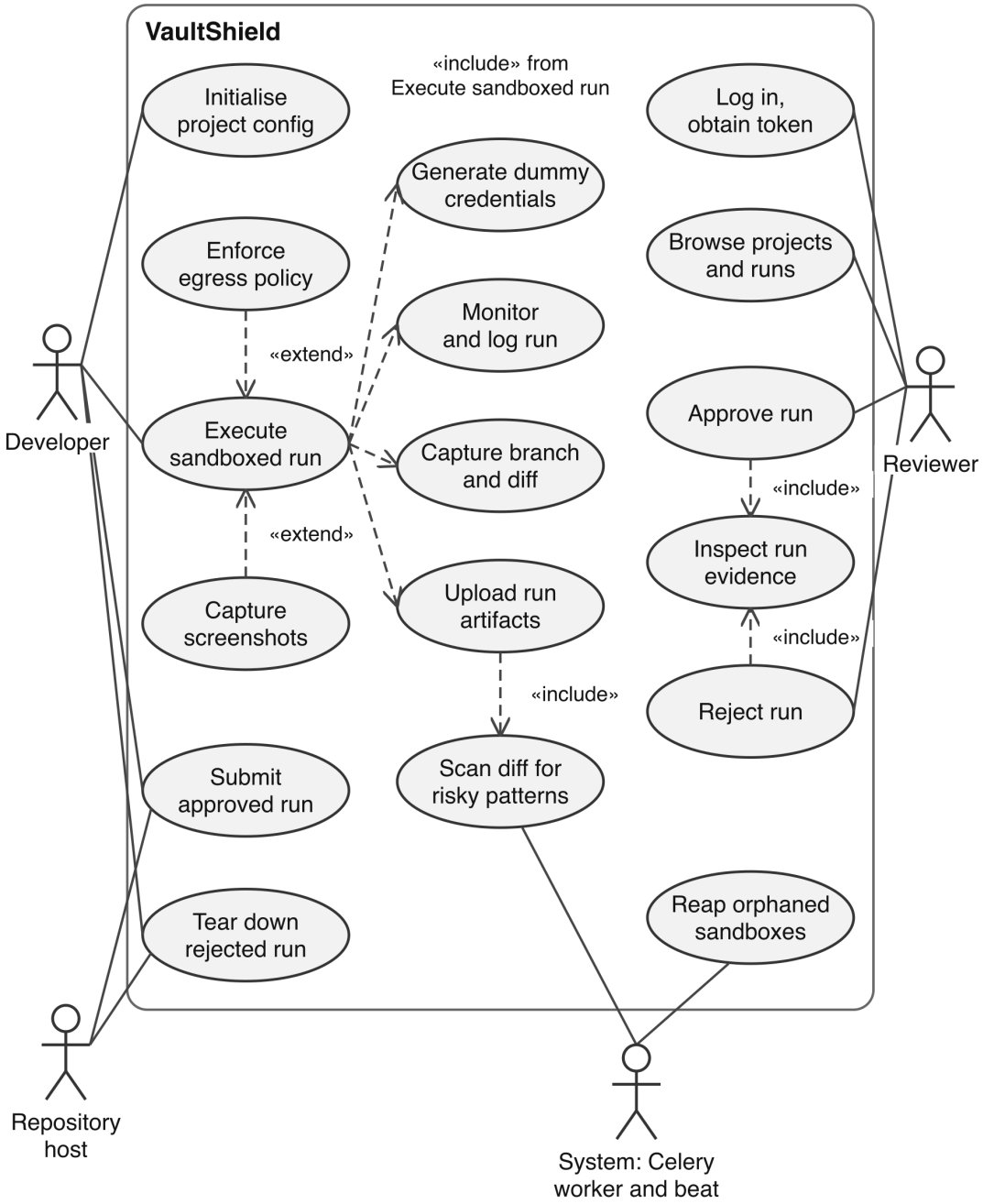


Figure 3.8: Use case diagram

3.4.4 Sequence Diagram

Figure 3.9 shows the message exchange for one approved run. Messages 1 to 9 cover the run, 10 and 11 the review, and 12 to 16 the submission. The backend learns of the run before Docker creates anything, the branch and diff are produced by a different container from the one the agent controlled, and submit re-reads the status immediately before pushing.

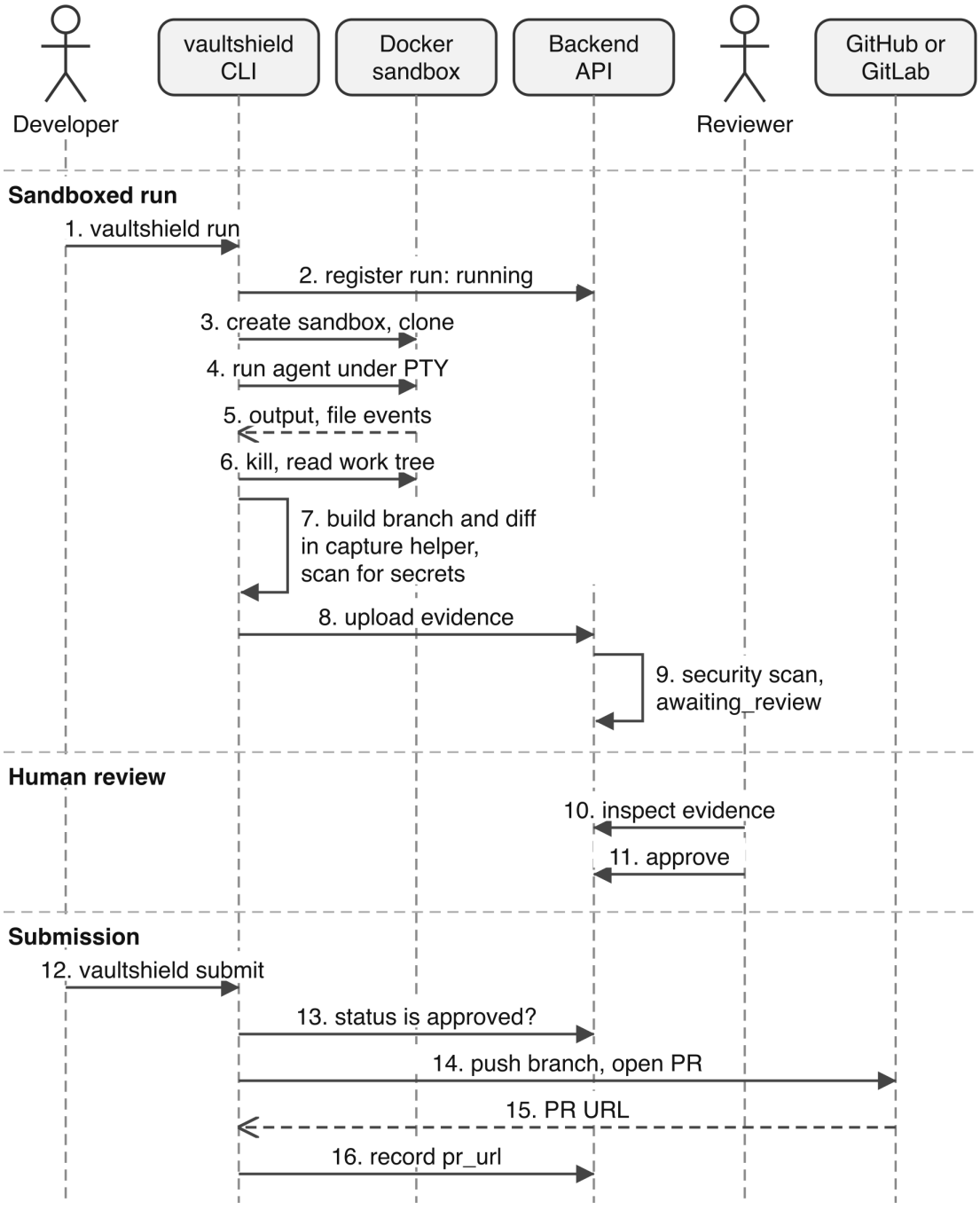


Figure 3.9: Sequence diagram of one complete approved run

Chapter 4

Project Planning and Cost Estimation

4.1 Project Development Methodology

VaultShield was developed with an iterative and incremental methodology based on specification-driven development. The team first wrote a requirements specification in which every requirement carried numbered acceptance criteria, and a design document with the architecture, data model and numbered correctness properties. The design was split into small, testable tasks, and each short iteration implemented a slice of tasks together with unit and property-based tests and, where isolation was involved, a Docker-backed integration test. The major phases were:

1. **Problem Study and Feasibility Spike:** survey the literature and measure how a production agent handles .env secrets under four protection configurations.
2. **Requirements and System Design:** write numbered requirements and define the architecture, run lifecycle, data model, REST API and threat model.
3. **CLI Core and Evidence Capture:** implement configuration, credential shaping, the sandbox lifecycle, agent adapters, monitoring, screenshots and Git capture.
4. **Backend and Dashboard:** implement the models, REST API, authentication, security scan, scheduled tasks and the review dashboard.
5. **Submission and Hardening:** implement PR and MR submission and teardown, then add the egress proxy, the trusted capture helper and token hygiene.
6. **Migration and Protect Module:** move the backend to MongoDB with transactional approval, and add the Protect module.
7. **Evaluation and Documentation:** run the tests and performance benchmark and write this report.

4.2 Project Schedule and Gantt Chart

The Semester VII activities follow the project Gantt chart, reproduced as Figure 5.16, and the Semester VIII activities are planned extensions. Table 4.1 lists each activity with its duration and target period.

Table 4.1: Planned project schedule for VaultShield

Sr.	Project activity	Duration	Target period
Semester VII			
1	Group formation and topic finalisation; identifying the scope and objectives of the project	1 week	10 Jul – 17 Jul 2026
2	Discussing the project topic with the help of a paper prototype	1 week	17 Jul – 24 Jul 2026
3	Identifying the functionalities of the project as per the objectives identified	1 week	24 Jul – 31 Jul 2026
4	Literature review and research paper study	1 week	31 Jul – 7 Aug 2026
5	Designing the graphical user interface	3 weeks	7 Aug – 28 Aug 2026
6	Presentation I	1 day	4 Sep 2026
7	Policy logic development	1 week	18 Sep – 25 Sep 2026
8	Core model development and training	1 week	25 Sep – 1 Oct 2026
9	Presentation II	1 day	9 Oct 2026
10	Incorporating the suggestions from Presentation II and submitting a draft copy to the project guide for proofreading	1 week	16 Oct – 23 Oct 2026
11	Draft submission to the guide for review	2 days	23 Oct – 24 Oct 2026
12	Final black book submission	1 week	26 Oct – 30 Oct 2026
13	External presentation and viva voce	1 day	2 Nov 2026
Semester VIII			
14	Evaluation with real agents on sample repositories	3 weeks	January 2027
15	Default sandbox hardening and fixes for issues found in testing	3 weeks	January – February 2027
16	Verification of the Codex and Kiro egress allowlists; reviewer-identity separation study	2 weeks	February 2027
17	Usability study with reviewers and dashboard refinements	2 weeks	March 2027
18	Research paper drafting and submission	2 weeks	March 2027
19	Final report, demonstration and viva preparation	3 weeks	March – April 2027

4.3 Team Roles and Responsibilities

The team of four divided responsibilities by architectural plane, with shared ownership of testing and documentation, as recorded in Table 4.2.

Table 4.2: Team roles and responsibilities

Member	Primary role	Responsibilities
Aayush Subhash Balip	Backend and data engineering	Django REST API, MongoDB models, run state machine, artifact ingest, Celery tasks, Docker Compose stack
Krish Mahesh Patil	Frontend and user experience	React dashboard, run detail tabs, decision controls, accessibility and responsive layout
Muhammad Anis Momin	Security research and sandbox engineering	Sandbox lifecycle, credential shaping, egress proxy, trusted capture, token hygiene, threat model
Sameer Hasan Saiyed	Integration, testing and documentation	CLI pipeline, agent adapters, submit and teardown, test strategy, benchmark and report

4.4 Software Size Estimation

4.4.1 KLOC-Based Estimation

The size of the delivered software was measured directly from the repository as logical source lines, excluding blank lines, comments, docstrings, migrations, lock files and generated content. Table 4.3 reports the counts.

Table 4.3: Measured software size by component

Component	Product code, SLOC	Test code, SLOC
CLI package and tools	12,404	16,697
Backend	1,150	2,552
Frontend	4,356	2,511
Deployment configuration	160	–
Total delivered product	**18,070**	**21,760**

The delivered product therefore measures KLOC = 18.07. Test code is excluded from the primary estimate, because COCOMO sizes delivered source instructions.

4.5 Effort Estimation Using COCOMO

4.5.1 Basic COCOMO Model

The Basic Constructive Cost Model [15] estimates effort and schedule from size alone. VaultShield was classified as an organic project, built by a small team in a familiar environment with flexible requirements, so the coefficients are a = 2.4, b = 1.05, c = 2.5 and d = 0.38. E is effort in person-months, T is development time in months and P is the average number of persons.

$E = a \times (KLOC)^{b} = 2.4 \times (KLOC)^{1.05}$
$T = c \times (E)^{d} = 2.5 \times (E)^{0.38}$
$P = E / T$

4.5.2 Calculation

With KLOC = 18.07, $(18.07)^{1.05} = 20.884$, so $E = 2.4 \times 20.884 = 50.12$ person-months. Then $(50.12)^{0.38} = 4.426$, so $T = 2.5 \times 4.426 = 11.06$ months, and $P = 50.12 / 11.06 = 4.53$ persons. Table 4.4 summarises the estimate.

Table 4.4: Basic COCOMO estimate

Quantity	Formula	Value for KLOC = 18.07
Effort E	$2.4 \times KLOC^{1.05}$	50.12 person-months
Development time T	$2.5 \times E^{0.38}$	11.06 months
Average staffing P	E / T	4.53 persons

The estimate of about 4.5 persons over about 11 months agrees closely with the actual four-member team working across two semesters, which supports the organic classification.

4.6 Human Resource Cost

The project was completed by the four team members as part of their academic programme, and no salary, stipend or consultant fee was paid. Table 4.5 therefore records actual cash expenditure rather than a hypothetical value for unpaid effort.

Table 4.5: Actual human-resource cash expenditure

Cost item	**Actual basis**	**Actual cost in ₹**
Student development effort	Unpaid academic project work	0
Guide supervision and review	No project-specific payment	0
Total		**0**

4.7 Hardware, Software and Cloud Infrastructure Cost

The work used existing team laptops, open-source software and local infrastructure, and no paid cloud environment was commissioned, as shown in Table 4.6.

Table 4.6: Actual hardware, software and infrastructure cash expenditure

Category	**Actual basis**	**Cost in ₹**
Hardware	Existing team laptops; no new equipment	0
Software	Python, Django, React, Git, Docker and test frameworks, all open source	0
Database and hosting	Local MongoDB replica set and free-plan repository hosting	0
Cloud and model APIs	No paid deployment or project-funded usage charge	0
Total		**0**

4.8 Testing, Deployment and Maintenance Cost

Testing and the MVP deployment ran on local machines with Docker, so no testing or deployment cost was incurred, as shown in Table 4.7. Future maintenance is not included because it has not been incurred.

Table 4.7: Actual testing, deployment and maintenance cash expenditure

Item	**Actual basis**	**Actual cost in ₹**
Automated and Docker-backed tests	Local machines and open-source frameworks	0
MVP deployment	Local Docker Compose stack	0
Total		**0**

4.9 Total Estimated Project Cost

Table 4.8 consolidates the actual project-specific cash expenditure.

Table 4.8: Total actual project cash expenditure

Cost head	**Source**	**Actual cost in ₹**
Human resources	Table 4.5	0
Hardware, software and infrastructure	Table 4.6	0
Testing, deployment and maintenance	Table 4.7	0
Total actual project cash expenditure		**0**

4.10 Assumptions and Cost Justification

Only amounts paid specifically for the project are included. Unpaid student effort, depreciation of equipment already owned, normal utilities and possible future maintenance are not given hypothetical monetary values, and tools already available at no extra charge count as zero. The actual cash expenditure for VaultShield is therefore ₹0. This basis is

separate from the COCOMO estimate, which describes effort and schedule rather than money spent.

4.11 Risk Assessment and Mitigation

Table 4.9 lists the main technical, security and project risks with their likelihood, impact and mitigation.

Table 4.9: Risk assessment and mitigation

Risk	Likelihood	Impact	Mitigation
Container escape through a kernel or Docker vulnerability	Low	Very high	Docker socket never mounted; only a dummy credential file is mounted; gVisor or microVM isolation planned
Malicious dependency running install hooks	Medium	High	Installs only inside the disposable sandbox with dummy credentials; egress allowlist available
Network exfiltration when egress control is off	Medium	High	Log-only and allowlist modes through a hardened proxy; default-deny egress planned
Real secrets reaching the sandbox or artifacts	Low	Very high	Dummy credentials, one-shot clone credential, token-hygiene scan with diff quarantine
Agent tampering with Git state to hide changes	Low	High	Trusted capture in a separate helper from a pre-agent base
Approval gate bypassed by a token holder	Medium	Medium	Single-developer trust domain documented; reviewer identities planned
Upstream agent CLI changes break an adapter	High	Medium	Adapter interface isolates changes; mock adapter keeps the pipeline testable
Schedule slippage or member unavailability	Medium	Medium	Working pipeline after every iteration; shared module knowledge through testing

Chapter 5

Project Implementation

5.1 Code Snippets

5.1.1 Launching the Sandbox Container

The first snippet is the core of Sandbox.create. Before any Docker call it checks that the credential file exists, because Docker silently creates an empty directory at a missing bind-mount source. Egress options are passed only when they are set, and any failure removes the partial container and raises a single SandboxError. As Figure 5.1 shows, the only host path given to the container is the read-only credential file, and teardown force-removes the container from a finally block so that it runs on every exit path.

Sandbox container creation and teardown

```
# Sandbox.create (body, docstring elided)
try:
    if not env_file.is_file():
        raise SandboxError(
            f"Sandbox creation failed for run {self.run_id}: "
            f"credential file not found at {env_file}"
        )

    self._ensure_image(base_image)

    egress_kwargs = {
        key: value
        for key, value in (
            ("network", network),
            ("cap_drop", cap_drop),
            ("sysctls", sysctls),
            ("extra_hosts", extra_hosts),
        )
        if value is not None
    }
    self.container = self._client.containers.create(
        base_image,
        command=_KEEP_ALIVE_CMD,                # ["tail", "-f", "/dev/null"]
        name=container_name(self.run_id),        # "vaultshield-run-<run_id>"
        working_dir=_WORKDIR,
        volumes={
            str(env_file.resolve()): {
                "bind": _ENV_MOUNT_PATH,         # /workspace/.env.sandbox
                "mode": "ro",
            }
        },
        ports=(
            {f"{publish_port}/tcp": None} if publish_port is not None else None
        ),
        **egress_kwargs,
    )
    self.container.start()

    self._clone_repo(repo_url, resolve_credential=resolve_credential)
    self._install_dependencies(project_dir)
except Exception as exc:
    self._cleanup_after_failure()
    if isinstance(exc, SandboxError):
        raise
    raise SandboxError(
        f"Sandbox creation failed for run {self.run_id}: {exc}"
    ) from exc

# Sandbox.teardown (docstring elided): idempotent, kills every exec'd process
def teardown(self) -> None:
    if self.container is None:
        return
    try:
        self.container.remove(v=True, force=True)
    except docker.errors.NotFound:
        pass
    finally:
        self.container = None
```

Figure 5.1: Sandbox container creation and teardown

5.1.2 One-Shot Credential for Private Clones

The second snippet clones a private repository without giving the agent the repository-host token. The token is placed only in the environment of the single git clone exec, and the command line names environment variables rather than containing their values. As Figure 5.2 shows, the token is therefore absent from the remote URL, the process arguments, the container environment and error messages, and its lifetime is exactly the duration of the clone.

Authenticated clone with a one-shot credential helper

```
# Sandbox._clone_repo (docstring elided)
provider = _clone_credential_provider(repo_url)      # strict github/gitlab
allowlist
credential, unavailable_reason = self._clone_credential(
    provider, resolve_credential
)

if credential is not None:
    from vaultshield.submit import git_credential_args

    assert provider is not None  # implied by credential is not None
    config_args, env = git_credential_args(credential, provider)
    self._exec_or_raise(
        # `config_args` names the two environment variables the helper
        # reads; the token itself is only ever in `env`.
        ["git", *config_args, "clone", repo_url, _REPO_DIR],
        workdir=_WORKDIR,
        step="git clone",
        environment=env,                 # applies to this one exec only
        secrets=(credential.token,),     # redacted from any error message
    )
    return

try:
    self._exec_or_raise(
        ["git", "clone", repo_url, _REPO_DIR],
        workdir=_WORKDIR,
        step="git clone",
        environment={_GIT_TERMINAL_PROMPT_ENVVAR: "0"},
    )
except SandboxError as exc:
    if unavailable_reason is None:
        raise
    raise SandboxError(
        f"Sandbox creation failed for run {self.run_id} during git "
        f"clone: {repo_url} could not be cloned, and no usable "
        f"{provider} credential was available to authenticate it. If "
        f"the repository is private, {unavailable_reason} "
        f"Original git failure: {exc}"
    ) from exc
```

Figure 5.2: Authenticated clone with a one-shot credential helper

5.1.3 Event Logging and Off-Limits Enforcement

The third snippet, in Figure 5.3, shows the event record type, the off-limits path matcher and the append-only JSON Lines writer. Every record has exactly three fields matching the backend's RunEvent model, and json.dumps keeps each record on one line, so a partially written log stays readable. The monitor raises OffLimitsViolation only after the writer has run.

Event records, off-limits matching and the JSON Lines writer

```
@dataclass(frozen=True)
class EventLogRecord:
    event_type: Literal["output", "file_change", "violation"]
    timestamp: str  # ISO-8601
    payload: str


def _match_off_limits(
    changed_path: str,
    workdir: Path,
    entries: tuple[str, ...],
) -> str | None:
    # (docstring elided) returns the first violated entry, or None
    if not entries:
        return None
    candidates = _path_candidates(changed_path, workdir)
    for entry in entries:
        for candidate in candidates:
            if candidate == entry or candidate.startswith(entry + "/"):
                return entry
            if fnmatch.fnmatch(candidate, entry):
                return entry
    return None


def _append_event_log(path: Path, records: Iterable[EventLogRecord]) -> Path:
    # (docstring elided) append-only: an existing log is never rewritten
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("a", encoding="utf-8", newline="\n") as log_file:
        for record in records:
            log_file.write(json.dumps(asdict(record), ensure_ascii=False) + "\n")
    return path
```

Figure 5.3: Event records, off-limits matching and the JSON Lines writer

5.1.4 Egress Allow/Deny Decision

The fourth snippet, in Figure 5.4, is the egress proxy's decision function, evaluated for every CONNECT request and TLS connection. A host that is not on the allowlist is never resolved in allowlist mode, IP literals must match an IP entry exactly, and every resolved address must be publicly routable, so an allowlisted name cannot be pointed at an internal address. In log-only mode only the allowlist rule is relaxed.

Egress decision for one request target

```
def decide(
    entries: Sequence[AllowEntry],
    mode: str,
    host: str,
    port: int,
    resolver: Resolver = default_resolver,
    *,
    address_policy: Callable[[str], bool] = is_public_address,
) -> Decision:
    norm = normalise_host(host)
    if norm is None or not isinstance(port, int) or not 1 <= port <= 65535:
        return Decision("deny", "bad_request", host if norm is None else norm,
                        port if isinstance(port, int) else 0)

    if _ip_or_none(norm) is not None:
        if not host_listed(entries, norm):
            return Decision("deny", "ip_literal", norm, port)
        if not matches(entries, norm, port):
            return Decision("deny", "port_not_allowed", norm, port)
        if not address_policy(norm):
            return Decision("deny", "private_address", norm, port)
        return Decision("allow", None, norm, port, (norm,))

    verdict = "allow"
    reason: str | None = None
    if not host_listed(entries, norm):
        if mode != "log-only":
            return Decision("deny", "not_in_allowlist", norm, port)
        verdict, reason = "would_deny", "not_in_allowlist"
    elif not matches(entries, norm, port):
        return Decision("deny", "port_not_allowed", norm, port)

    addresses = _resolve_with_timeout(resolver, norm, port)
    if not addresses:
        return Decision(verdict, reason, norm, port, ())
    if not all(address_policy(address) for address in addresses):
        return Decision("deny", "private_address", norm, port)
    ordered = sorted(addresses, key=lambda a: 0 if ":" not in a else 1)
    return Decision(verdict, reason, norm, port, tuple(dict.fromkeys(ordered)))
```

Figure 5.4: Egress decision for one request target

5.1.5 Branch Commit and Structured Diff

The fifth snippet, in Figure 5.5, runs inside the trusted capture helper. commit_run_branch refuses unsafe repository configuration and creates exactly one commit, using --allow-empty so that a run which changed nothing still produces a reviewable result and --no-verify so that no repository hook runs. build_structured_diff disables rename detection, external diff programs and text conversion filters.

Committing the run branch and computing the diff

```
def commit_run_branch(sandbox: Sandbox, run_id: str) -> str:
    branch = run_branch_name(run_id)              # "vaultshield/run-<run_id>"

    # Refuse a gitfile/symlinked `.git` and any filter/diff driver
    # BEFORE `git add`, which is what would run a filter.
    _assert_capture_repo_is_safe(sandbox)

    _exec_or_raise(sandbox, _git("checkout", "-b", branch),
                   step=f"creating branch {branch}")
    _exec_or_raise(sandbox, _git("add", "--all"),
                   step="staging working-tree changes")
    _exec_or_raise(
        sandbox,
        _git("-c", f"user.name={COMMITTER_NAME}",
             "-c", f"user.email={COMMITTER_EMAIL}",
             "commit", "--allow-empty", "--no-verify",
             "-m", _commit_message(run_id)),
        step=f"committing changes to {branch}",
    )
    return branch


# build_structured_diff (excerpt): per-file counts, then size-capped patches
result = _exec_or_raise(
    sandbox,
    _git("diff", "--numstat", "-z", "--no-renames",
         "--no-ext-diff", "--no-textconv", base_ref, branch),
    step=f"diffing {base_ref}..{branch}",
)
records = _parse_numstat(_raw_stdout(result))
metadata_cost = _metadata_cost(records)
if metadata_cost > MAX_DIFF_REQUEST_BYTES:          # 2 MiB upload budget
    raise GitCaptureError(
        "Git capture failed while sizing diff.json: the entries for "
        f"{len(records)} changed files take {metadata_cost} bytes even "
        f"with every patch left out, over the {MAX_DIFF_REQUEST_BYTES}-byte "
        "upload cap (MAX_DIFF_REQUEST_BYTES), so the backend would reject "
        "the diff"
    )
```

Figure 5.5: Committing the run branch and computing the diff

5.1.6 Approval State Machine

The sixth snippet, in Figure 5.6, is the inner function of apply_decision. The status check and status write are a single conditional update, and the Approval record is created in the same MongoDB transaction, so only one of two concurrent decisions can match. The loser receives HTTP 409 naming the run's actual status; before the retry wrapper was added, 54 of 72 losing calls in a concurrency test received a server error instead.

Transactional approve/reject decision

```
ALLOWED_DECISIONS_FROM = frozenset({"awaiting_review"})

def apply_decision(run: Run, decision: Decision) -> Run:
    # (docstring elided)
    decided_at = to_mongo_precision(timezone.now())

    def record_decision() -> None:
        with transaction.atomic():          # django_mongodb_backend's atomic()
            recorded_decision = (
                Approval.objects.filter(run=run)
                .values_list("decision", flat=True).first()
            )
            if recorded_decision is not None:
                raise StateTransitionError(
                    _stored_status_of(run), recorded_decision=recorded_decision
                )

            transitioned = Run.objects.filter(
                pk=run.pk, status__in=ALLOWED_DECISIONS_FROM
            ).update(status=decision, updated_at=decided_at)

            if not transitioned:
                raise StateTransitionError(_stored_status_of(run))

            Approval.objects.create(run=run, decision=decision)

    # Retried on MongoDB's TransientTransactionError only. "Measured at 54
    # spurious errors out of 72 losing calls before this was added."
    retry_on_transient_transaction_error(record_decision)

    run.status = decision
    run.updated_at = decided_at
    return run
```

Figure 5.6: Transactional approve/reject decision

5.2 Steps to Access the System

The system requires Python 3.11 or later, a running Docker daemon, Git, Node.js 18 or later, a MongoDB replica set or Atlas cluster, and Playwright Chromium. The commands assume macOS or Linux.

1. Clone the repository and copy .env.example to .env, setting VAULTSHIELD_ MONGODB_URL to the MongoDB connection string.
2. Start the backend and dashboard with docker compose up –build. The API is served on http://localhost:8000 and the dashboard on http://localhost:5173.
3. Create a user with docker compose exec web python manage.py createsuperuser.
4. Install the CLI:
   cd cli && python3 -m venv .venv && source .venv/bin/activate && pip install -e ".[dev]" && playwright install chromium
5. Authenticate with vaultshield login –backend-url http://localhost:8000 –username <user>, create a project in the dashboard and note its identifier.
6. In a checkout of the target repository, run vaultshield init to write .vaultshield.yml.

7. Execute a sandboxed run, for example with the mock agent:
   vaultshield run –agent=mock –prompt=”add a file” –repo-url=”$REPO” –project ”$PROJECT_ID”
8. Open the dashboard, select the run, inspect the Diff, Timeline, Screenshots and Security tabs, and choose Approve or Reject.
9. For an approved run, run vaultshield submit <run-id> to open the pull or merge request; for a rejected run, run vaultshield teardown <run-id>.

5.3 System Implementation

The dashboard screenshots below were captured by the repository’s layout tool, which serves the built dashboard with fixture data, so they show the real interface with illustrative data.

5.3.1 Command-Line Interface

The CLI provides the pipeline commands init, login, run, submit and teardown and the supporting commands protect and exec. During vaultshield run, the terminal prints one line per pipeline milestone, as shown in Figure 5.7, and each run writes a run directory containing the dummy credential file, events.jsonl, diff.json, the branch bundle and the screenshots.

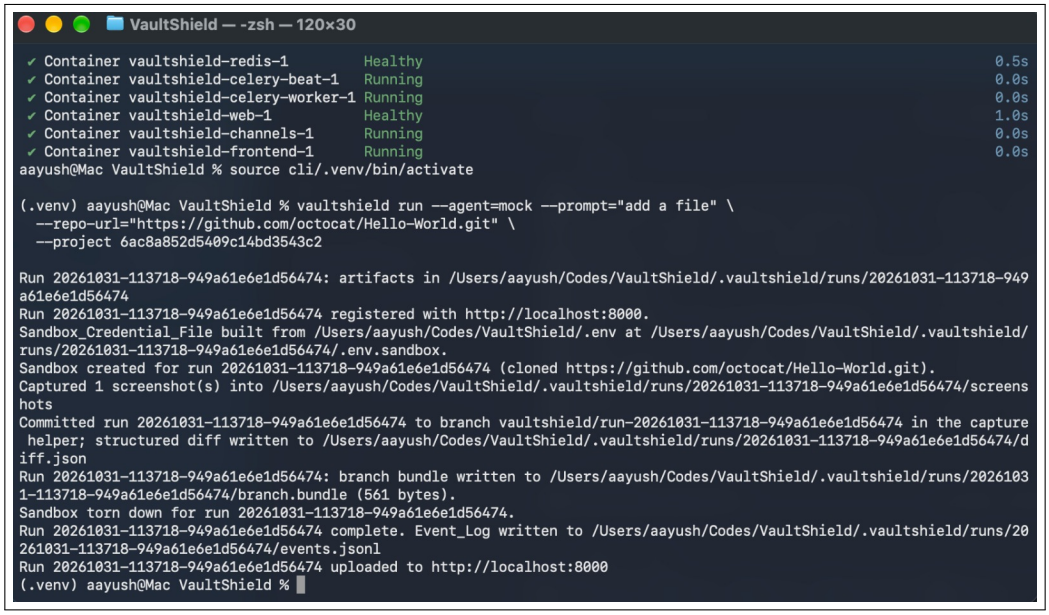


Figure 5.7: Terminal output of a successful vaultshield run with the mock agent

5.3.2 Landing Page and Projects Dashboard

The landing page in Figure 5.8 introduces the product with a one-line installer command and links to sign in. After sign-in, the projects overview in Figure 5.9 shows one card per project with its repository URL, a sidebar of projects and a New project button.

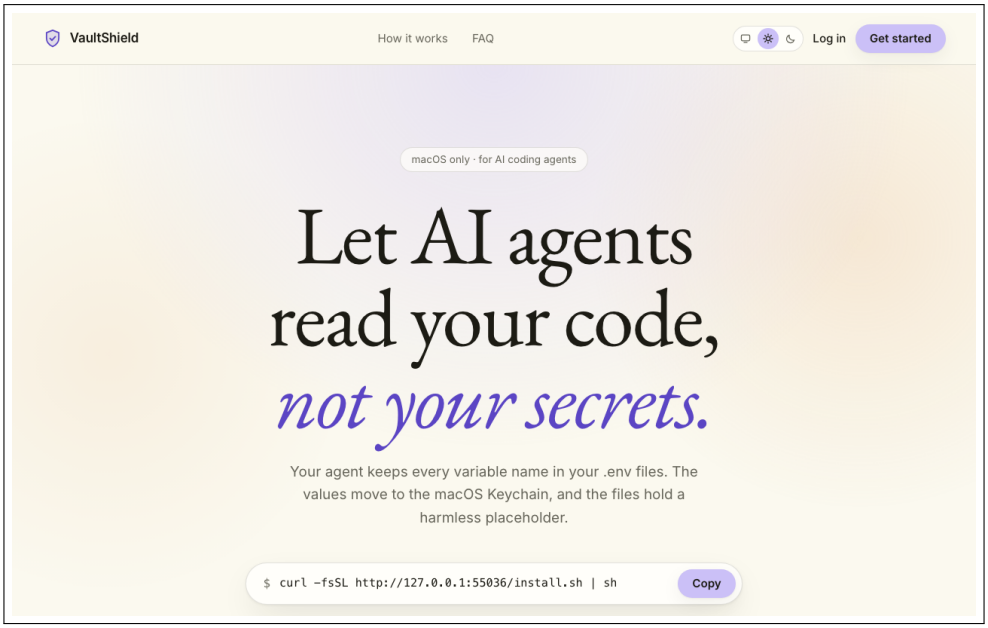


Figure 5.8: Landing page

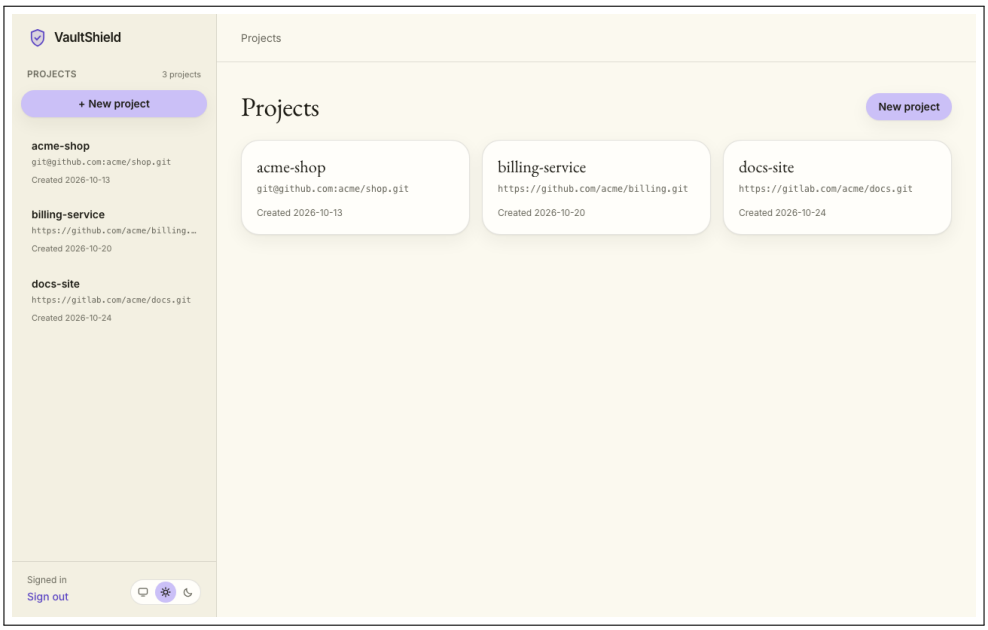


Figure 5.9: Projects dashboard

5.3.3 Run History

The run history in Figure 5.10 lists a project's runs newest first with the run identifier, agent, status badge and creation time, with status filters, search by run identifier and pagination.

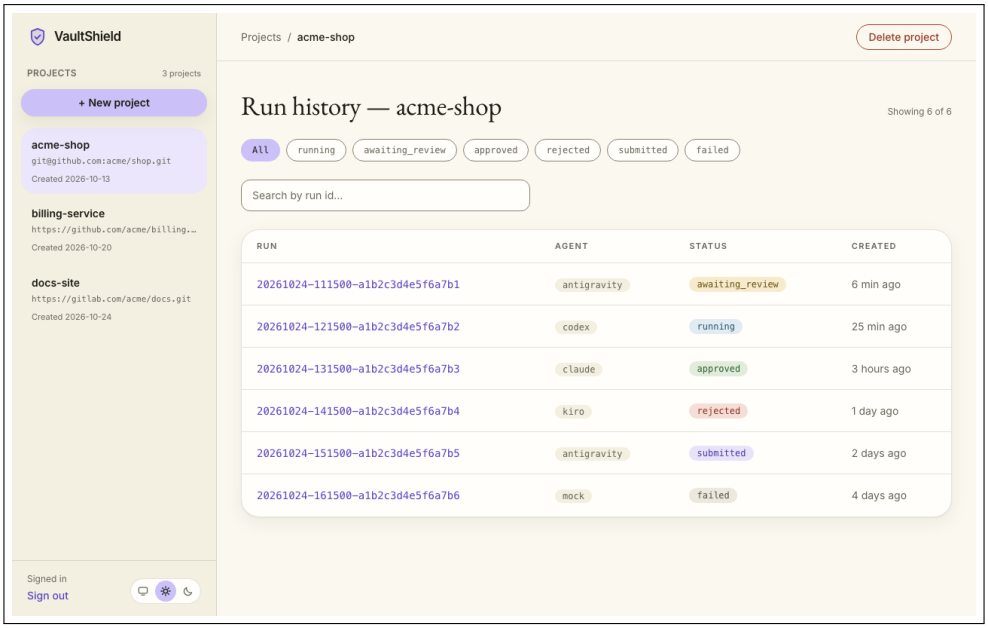


Figure 5.10: Run history with status filters

5.3.4 Run Detail Views

The run detail page has a header with the run's status, a decision banner while the run is awaiting review, and four tabs. The Diff tab in Figure 5.11 lists every changed file with its addition and deletion counts. The Timeline tab in Figure 5.12 shows the event log in order, giving the reviewer a view of what the agent did as well as what it changed [14]. The Screenshots tab in Figure 5.13 shows one image per configured route, and the Security tab in Figure 5.14 lists the findings of the automated scan with their rule, file and line. Findings inform the reviewer but do not block approval.

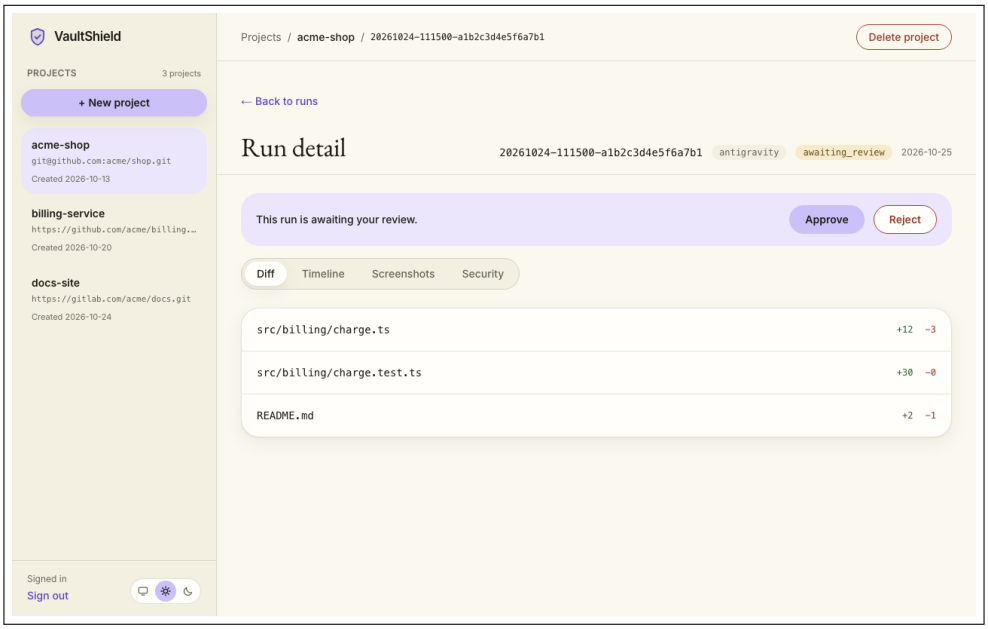


Figure 5.11: Run detail – Diff tab

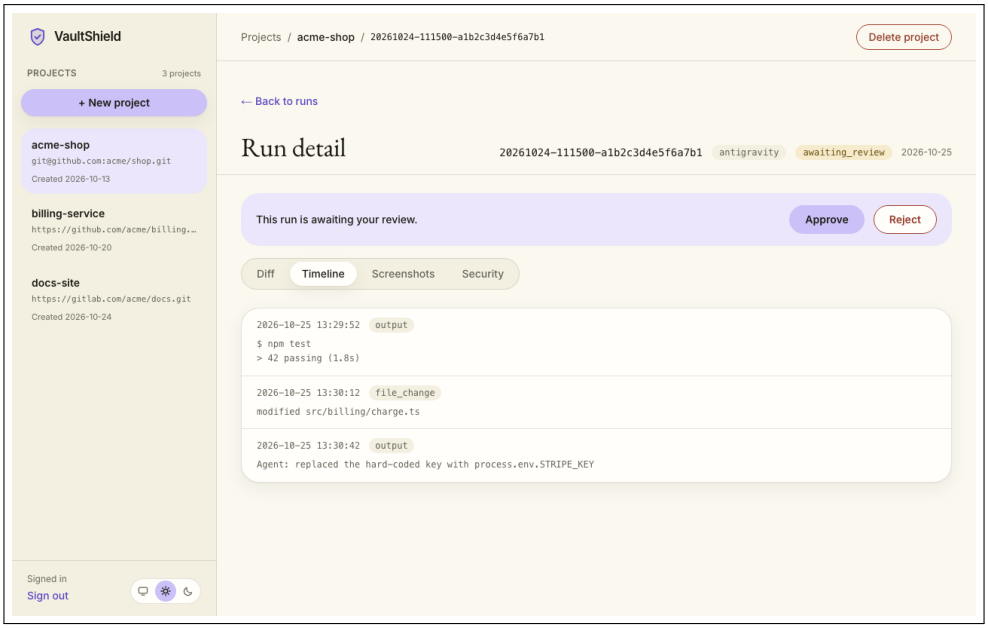


Figure 5.12: Run detail – Timeline tab

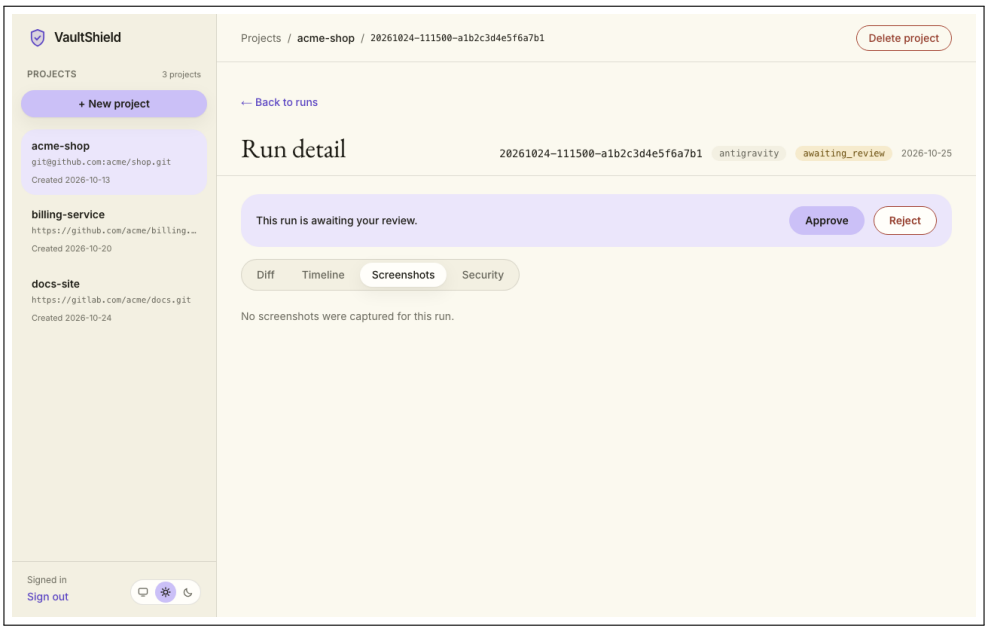


Figure 5.13: Run detail – Screenshots tab

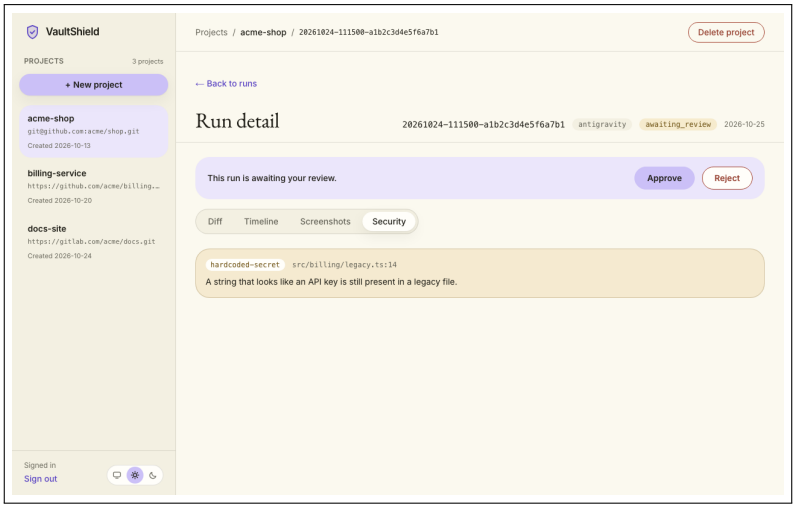


Figure 5.14: Run detail – Security tab

5.3.5 Approval Decision and Submission

When the reviewer approves a run, the buttons are replaced by the status “approved, awaiting submission” and the exact vaultshield submit command, as shown in Figure 5.15, because the backend never opens a pull request on its own.

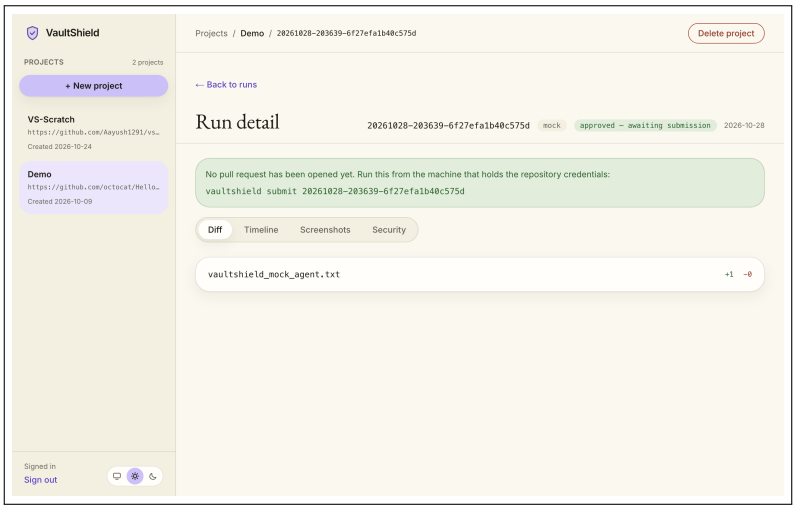


Figure 5.15: Run detail after approval, showing the submit command

5.3.6 Implementation Summary

The CLI creates the sandbox, monitors the agent, captures the change on a dedicated branch and uploads the evidence, while the dashboard presents that evidence and records the reviewer’s decision. Producing a change and accepting it therefore remain separate steps. Everything in this chapter was implemented in Semester VII: the CLI and adapters, the Docker sandbox with a fresh clone, dummy and one-shot credentials, egress control, monitoring with the event log and off-limits paths, trusted capture and the structured diff, the MongoDB backend and dashboard, security scanning, and approval-gated submission. The Agentic AI Orchestrator, stronger sandbox isolation, improved monitoring and role-based approval are planned for Semester VIII.

Two further captures are genuine executions. Capture (i) shows submit refusing a run that was still awaiting review, and no sandbox, helper or proxy left after the run. Capture (ii) is from the isolation probe of 9 October 2026, run against a sandbox in which the Antigravity agent had been signed in: the agent's credential files are inside the sandbox, direct network and Docker-socket paths fail, and the proxy records each host the agent contacted. No Antigravity task response was preserved, so none is shown.

Approval gate and cleanup check after the run, 1 Nov 2026

```
$ vaultshield submit 20261101-180424-c5458c45adecdf4a
Error: Run 20261101-180424-c5458c45adecdf4a is 'awaiting_review', not 'approved', so it cannot be su
bmitted. Nothing was pushed and no pull request was opened.
exit=1

$ docker ps -a --filter name=vaultshield-run --filter name=vaultshield-capture --filter name=vaultsh
ield-egress
NAMES     STATUS

$ docker network ls --filter name=vaultshield
NAME                 DRIVER
vaultshield_default   bridge

$ ls ../vaultshield_mock_agent.txt   # run from cli/: the host checkout root
ls: ../vaultshield_mock_agent.txt: No such file or directory
```

Execution capture (i): approval gate and cleanup check after a mock-agent run, 1 November 2026

Signed-in Antigravity agent in a VaultShield sandbox, 9 Oct 2026 (excerpt)

```
# ~/.vaultshield/probes/20261009T083128Z-egress-log-only.json (isolation_probe.py, 9 Oct 2026), sele
cted fields
agent: antigravity  egress: log-only  interactive_auth: True  https_proxy: http://vaultshield-egres
s:3128
  /root/.gemini/antigravity-cli/antigravity-oauth-token 497 bytes 0o600
  /root/.gemini/antigravity-cli/conversations/94aac82a-55f4-4de6-8658-3c7f2d5006ec.db 131072 bytes 0
o644
  /root/.gemini/antigravity-cli/log/cli-20261009_083130.log 38540 bytes 0o644
  probe internet TCP 1.1.1.1:443 -> unreachable
  probe DNS exfil (direct UDP) 8.8.8.8:53/udp -> unreachable
  probe file /var/run/docker.sock -> unreachable

# ~/.vaultshield/probes/20261009T094445Z-egress-allowlist-proxy.jsonl: first decision per host
09:44:47 deny  antigravity-unleash.goog                                    not_in_allowlist
09:44:47 deny  playwright.azureedge.net                                    not_in_allowlist
09:44:47 deny  playwright-akamai.azureedge.net                             not_in_allowlist
09:44:47 deny  playwright-verizon.azureedge.net                            not_in_allowlist
09:44:48 deny  antigravity-cli-auto-updater-974169037036.us-central1.run.app  not_in_allowlist
09:44:52 deny  play.googleapis.com                                         not_in_allowlist
09:45:03 allow oauth2.googleapis.com
09:45:04 deny  www.googleapis.com                                          not_in_allowlist
09:45:04 allow daily-cloudcode-pa.googleapis.com
... 29 decisions in total: 6 allowed, 23 denied
```

Execution capture (ii): probe and egress-proxy records from a sandbox with a signed-in Antigravity agent

5.4 Timeline Sem VII

The Gantt chart in Figure 5.16 shows the Semester VII activities across thirteen weeks, from group formation on 10 July 2026 to the external presentation and viva voce on 2 November 2026. The activities are grouped into four phases: project conception and initiation, design and implementation, evaluation and documentation, and final submission. All four members are owners of every activity, and every Semester VII activity is complete.

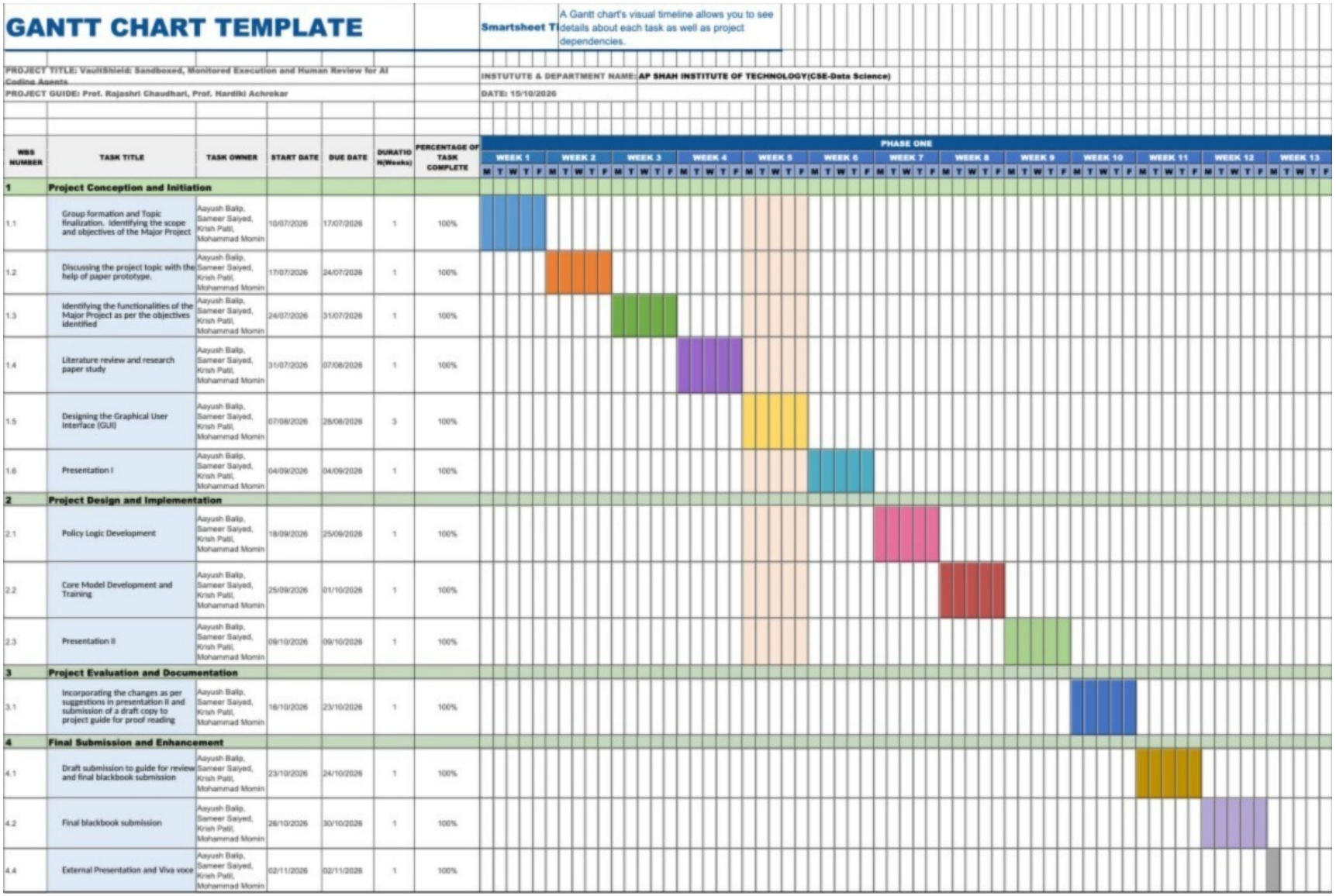


Figure 5.16: Gantt chart of the VaultShield major project

Chapter 6

Testing

6.1 Software Testing

VaultShield's value lies mostly in negative guarantees: the agent must not reach the host, real secrets must not enter the sandbox, a token must not appear in any artifact, and an unapproved run must not become a pull request. The team therefore traced every acceptance criterion to at least one test, expressed invariants as property-based tests with Hypothesis and fast-check, tested isolation claims against real Docker containers with a probing agent rather than mocks, and tested the backend against a real MongoDB replica set because the approval gate depends on genuine transaction semantics. Test fixtures redirect the home directory to an empty temporary directory so that no test can touch the developer's real files.

6.2 Types of Testing

Testing was organised into functional testing and system testing. Table 6.1 lists the test levels used within them.

Table 6.1: Types of testing performed

Level	Scope	Tools
Unit	Configuration parsing, credential shaping, off-limits matching, diff parsing, egress decisions, React components	pytest, Hypothesis, Jest, fast-check
Integration	CLI with real Docker, backend with MongoDB, upload with the API	pytest with Docker and a replica set
Security-focused	Isolation, secret suppression, network blocking, tamper resistance, approval atomicity	Probing agents in real containers, concurrency tests
System	The complete run, submit and teardown flows	Click CliRunner, Docker, timing harness
Usability and accessibility	Contrast ratios, responsive layout, keyboard navigation	Jest contrast test, Playwright layout tool

6.2.1 Functional Testing

Functional testing verified that each function, module and security control behaves as specified. Unit tests checked properties such as a dummy credential never containing the real value and the egress decision table for IP literals and private addresses. Security-focused tests checked that a non-allowlisted connection and DNS-based exfiltration are

blocked, that a tampered in-container Git cannot hide a file from the diff, and that concurrent approve and reject requests resolve to exactly one decision.

6.2.2 System Testing

System testing drove the real vaultshield run command end to end with a real container, clone, agent process, trusted capture and teardown, and the same harness executed the 40 timed runs reported in Chapter 7.

6.3 Testing Results and Findings

Table 6.2 summarises the results of the three automated suites.

Table 6.2: Automated test suite results

Suite	Tests	Passed	Failed	Pass rate
CLI	2,080	2,078	2	99.90%
Backend	341	341	0	100.00%
Frontend	283	283	0	100.00%
Total	**2,704**	**2,702**	**2**	**99.93%**

No test was skipped. The lowest text contrast was 4.77:1 in the light theme and 5.80:1 in the dark theme, above the 4.5:1 threshold, and the layout report recorded no horizontal overflow across 63 page and width combinations. Table 6.4 presents representative test cases, each corresponding to an automated test or to the benchmark harness.

Table 6.4: Sample test cases

ID	Description	Expected and actual result	Status
TC-01	Sandbox isolation	Probing agent cannot read the host .env, home directory or Docker socket; host checkout untouched in 40 of 40 runs	Pass
TC-02	Secret suppression	No generated dummy value contains its real value; 135 of 135 tests passed	Pass
TC-03	Network blocking	CONNECT to unlisted hosts, IP literals and the host answered 403; direct DNS lookups fail	Pass
TC-04	Off-limits enforcement	Write to a protected path records a violation event, then fails the run	Pass
TC-05	Tamper resistance	A file hidden by a tampered in-container Git still appears on the run branch	Pass
TC-06	Token hygiene	All five credential-leak vectors fail the run and quarantine the diff	Pass
TC-07	Approval gating	submit refuses every non-approved run with no push and no PR	Pass
TC-08	Concurrent decisions	Exactly one winner per trial and exactly one Approval record	Pass
TC-09	Monitoring completeness	Every agent write in the timeline; present in 32 of 40 runs, while the diff recorded 40 of 40	Partial
TC-10	Hanging sign-in is bounded	Expected to stop within 60 s; took 600.3 s	Fail

6.4 Sample Test Cases

Two of the 2,704 tests failed. Both are Docker-backed tests of the optional interactive sign-in transport, used only when the antigravity agent runs with --interactive-auth; the first is TC-10 in Table 6.4.

Failure 1: interactive sign-in timeout. The hanging sign-in test expects the CLI to give up within 60 s of a silent sign-in, but it took 600.33 s. The deadline is a socket timeout on the Docker exec stream, and on Docker Desktop 29.2.1 it does not interrupt the blocking read. This is a genuine availability defect in the optional authentication flow; it exposes no secret. The correction is an independent watchdog deadline that kills the exec.

Failure 1, excerpt of pytest output

```
$ pytest --durations=2 tests/test_sandbox_execution.py::<hanging sign-in test>
___ test_a_hanging_signin_hits_the_read_timeout_instead_of_blocking_forever ____
>           assert outcome is not None
E           assert None is not None
tests/test_sandbox_execution.py:1588: AssertionError
600.33s call     tests/test_sandbox_execution.py::test_a_hanging_signin_hits_the_read_timeout_instea
d_of_blocking_forever
FAILED tests/test_sandbox_execution.py::test_a_hanging_signin_hits_the_read_timeout_instead_of_block
ing_forever
```

Failure 1: hanging sign-in test, excerpt of the pytest output (1 November 2026)

Failure 2: redacted echo expectation. The stdin test expects the raw sentinel value in the terminal output, which shows RECEIVED:[***]. Redaction replaces only an exact match of the sent value, so the output proves that the value was delivered and then masked as designed. The failure is an outdated assertion, not failed credential delivery; the assertion must expect the redacted output.

Failure 2, excerpt of pytest output

```
$ pytest --durations=2 tests/test_sandbox_execution.py::<stdin delivery test>
______ test_exec_interactive_really_delivers_stdin_to_a_container_process ______
                handle.send_line(_STDIN_SENTINEL)
>               assert f"RECEIVED:[{_STDIN_SENTINEL}]" in joined, joined
E               AssertionError: ***
E                 RECEIVED:[***]
E               assert 'RECEIVED:[interactive-stdin-sentinel-b41c]' in '***\nRECEIVED:[***]'
tests/test_sandbox_execution.py:1230: AssertionError
0.21s call     tests/test_sandbox_execution.py::test_exec_interactive_really_delivers_stdin_to_a_con
tainer_process
FAILED tests/test_sandbox_execution.py::test_exec_interactive_really_delivers_stdin_to_a_container_p
rocess
```

Failure 2: stdin delivery test, excerpt of the pytest output (1 November 2026)

Reliability impact. The two failures represent about 0.07% of the 2,704-test campaign. One is a genuine availability defect in an optional authentication flow; the other is an outdated test expectation. Neither affects the default non-interactive workflow, and the security-critical isolation, credential substitution, egress, trusted capture, token-hygiene and approval tests passed.

6.5 Conclusion on Testing

Of 2,704 automated tests, 2,702 passed, and every security-critical behaviour was exercised against real Docker and MongoDB. The remaining gaps, the sign-in deadline and the file watcher's missed events, affect availability and supplementary evidence, not isolation or change capture.

Chapter 7

Result and Discussions

7.1 Introduction

This chapter evaluates VaultShield with a timing harness that executed 40 complete vaultshield run invocations against real Docker, a baseline of the same agent run directly on the host, the security-focused test cases of Chapter 6, the feasibility spike on secret exposure, and a case study of preserved run directories. All measurements used the deterministic mock agent, which runs inside the sandbox through the same path as a real agent but makes no model API calls, so the results isolate the pipeline's own overhead. The measurements were taken on an Apple MacBook Air with an M4 processor running Docker Desktop 29.2.1, with a small local repository and a stubbed backend.

7.2 Evaluation Metrics

Table 7.1 defines the metrics used.

Table 7.1: Evaluation metrics

Metric	Definition
End-to-end latency	Wall-clock time of one vaultshield run, from command start to exit
Stage latency	Time spent in each pipeline stage
Overhead	Difference between VaultShield latency and the same agent run directly on the host
Isolation effectiveness	Fraction of runs in which the host checkout was unchanged and host files were unreachable
Capture and monitoring completeness	Fraction of runs whose diff and event log contained the agent's change
Detection of unsafe actions	Fraction of injected unsafe behaviours that were blocked or detected
Cleanup reliability	Number of VaultShield containers or networks left after all runs

7.3 Performance Analysis

Table 7.2 reports the stage latency of the default configuration over 20 runs, in seconds, and Figure 7.1 shows the mean of each stage.

Table 7.2: Stage latency in the default configuration

Stage	Mean	Std. dev.	Share
Sandbox creation, clone and install	0.239	0.017	17.0%
Agent execution and monitoring	0.413	0.033	29.4%
Trusted capture	0.651	0.053	46.4%
Token-hygiene scan	0.019	0.008	1.4%
Teardown	0.042	0.006	3.0%
Other stages	0.039	–	2.8%
End-to-end total	**1.403**	**0.075**	**100%**

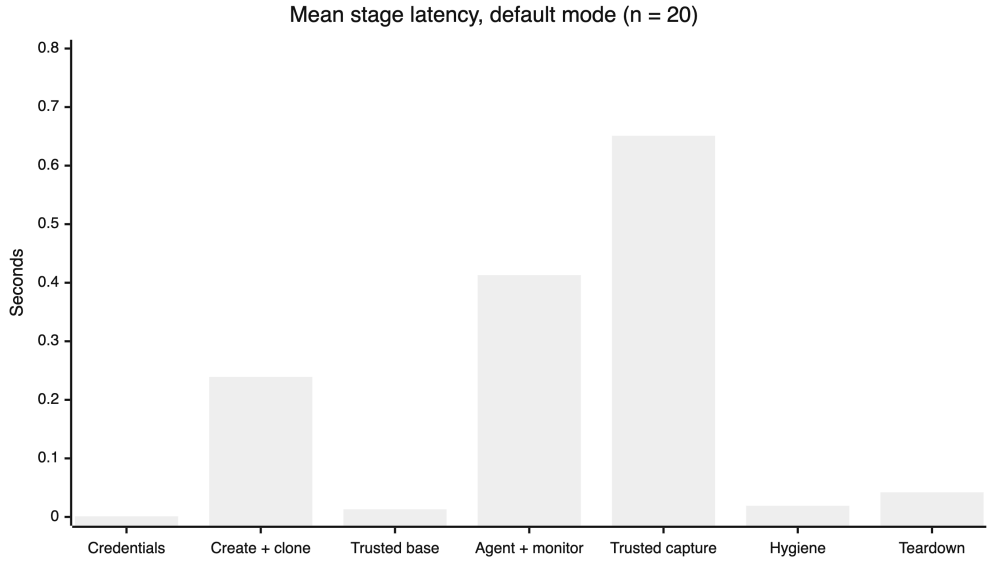


Figure 7.1: Mean stage latency in the default configuration

Trusted capture is the most expensive stage because it starts a second container, extracts the work tree and rebuilds the repository; this is the price of a diff the agent cannot falsify. Agent execution took 0.413 s against 0.186 s on the host, the difference covering the in-container watcher and Docker's exec API. Table 7.3 and Figure 7.2 compare the configurations.

Table 7.3: End-to-end latency by configuration

Configuration	Runs	Mean time, s
Direct on host: clone and agent	20	0.237
VaultShield default	20	1.403
VaultShield with egress allowlist	10	2.213
VaultShield with one screenshot route	10	3.294

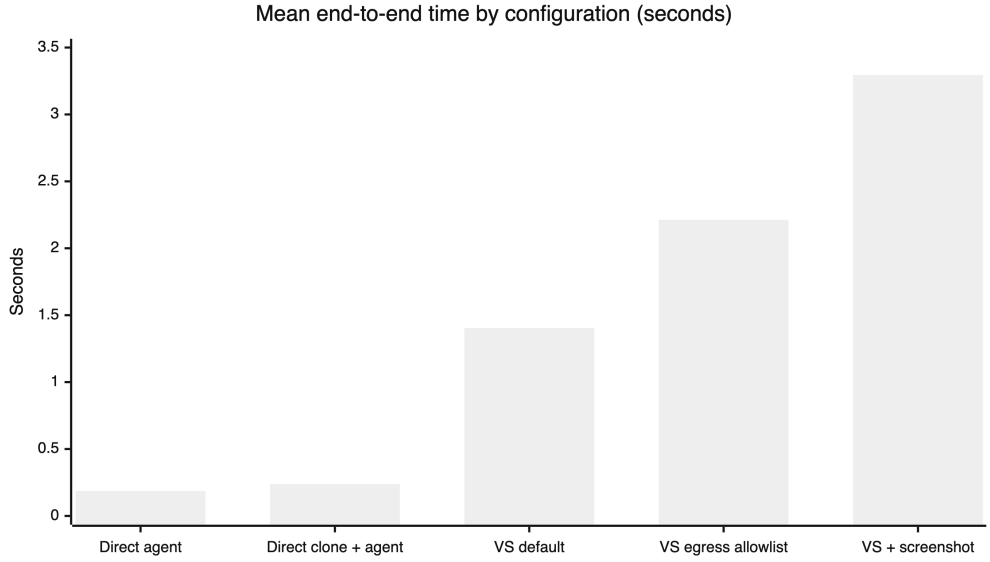


Figure 7.2: Mean end-to-end time by configuration

The egress allowlist added 0.810 s, mostly for creating the per-run networks and proxy and removing them at teardown. One screenshot added 1.891 s, almost all of it spent starting the application and waiting for it to answer.

7.4 Overhead with and without VaultShield

Against direct execution with a fresh clone, VaultShield's default mode added 1.166 s per run and the egress allowlist mode added 1.976 s. Because this overhead is fixed while a real agent session typically lasts minutes, it projects to 0.39% and 0.66% of a five-minute session respectively. Table 7.4 places the overhead alongside the qualitative differences.

Table 7.4: Running an agent with and without VaultShield

Dimension	Direct execution	VaultShield
Fixed overhead	–	+1.166 s, or +1.976 s with egress
Host checkout unchanged	No	Yes, in every run
Real .env values visible to the agent	Yes	No
Change captured on a reviewable branch	No	Yes, in every run
Outbound traffic restricted and logged	No	With egress allowlist
Approval required before a PR	No	Yes
Containers or networks left behind	–	0

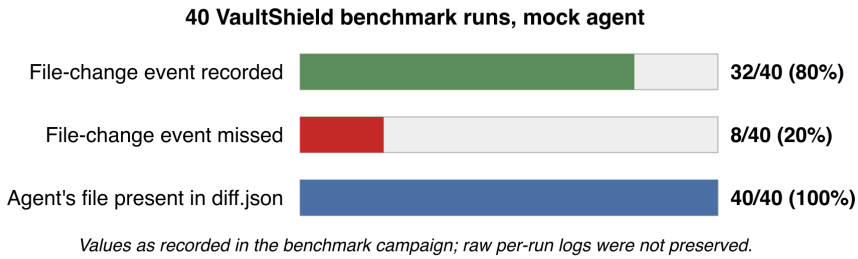
7.5 Safety Events Captured

Table 7.5 summarises the safety-relevant results across the 40 harness runs and the security-focused test cases. These are rates over injected scenarios, which show that each control works for the behaviour it was designed against but do not prove the controls are exhaustive.

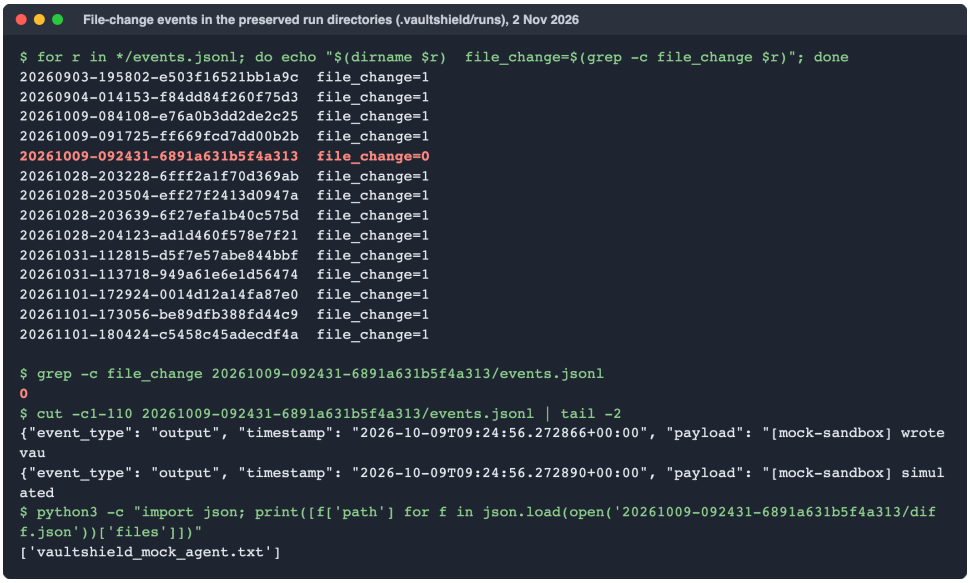
Table 7.5: Safety events and evidence captured

Scenario	Result
Host checkout unchanged after the run	40 of 40
Agent's file present in diff.json	40 of 40
File-change event recorded for the agent's file	32 of 40
Non-allowlisted connections blocked and logged	4 of 4
Credential-leak vectors detected	5 of 5
Tampered in-container Git defeated	2 of 2
Values in six real .env.sandbox files that are dummies	30 of 30
Containers and networks left after the campaign	0

File-change monitoring recorded the agent's write in 32 of the 40 VaultShield runs (80%), so 8 of 40 events were missed, while the agent's file was present in diff.json in all 40 runs. The in-container watcher polls the work tree, and the mock agent's write lands within milliseconds of the run ending, so a write can fall between two polls. The gap matters because the timeline is the reviewer's view of what the agent did: a missed event means a change that appears in the diff without a matching entry in the timeline. It does not mean an unrecorded change, because the trusted capture helper rebuilt every change from a pre-agent base. The diff is therefore the authoritative record and the event log is corroborating evidence; an event-driven watcher and a cross-check of the event log against the diff are planned for Semester VIII. The raw per-run logs of the 40-run benchmark were not preserved, so the summary below plots the recorded figures. The terminal capture after it is a real example of a missed event, taken from the 14 run directories preserved in the repository: one of them has no file-change event although its diff contains the agent's file.



Monitoring completeness in the 40 VaultShield benchmark runs (mock agent), plotted from the recorded figures



Terminal capture over the preserved run directories: run 20261009-092431 has no file-change event, but its diff.json contains the agent's file

7.6 Secret Exposure With and Without Protection

The feasibility spike asked a production coding agent to read .env and .env.local and report the exact values of two fake keys. Table 7.6 reports the outcome under four protection configurations.

Table 7.6: Secret exposure under four protection configurations

Config	Protection layers	Verdict
C0	None	Both secrets printed
C1	Agent deny rules for .env files	.env.local read into context
C2	C1 plus a hook denying shell access to .env	.env.local read into context
C3	C2 plus real values replaced by a placeholder on disk	Safe

Agent-level deny rules proved brittle, and only keeping the real value off disk was safe. This is why the sandbox receives dummy values and why the Protect module moves values into the macOS Keychain for agents run outside the sandbox.

7.7 Case Study: Analysis of Real Runs

The repository preserves five complete mock-agent runs from development. In run 20261009-091725-ff669fcd7dd00b2b, the event log records seven events: the agent confirms that its working directory is the sandbox clone, narrates its work over 157.9 ms, and the in-container watcher then independently reports the created file 106.5 ms after the agent's write. The diff contains exactly that file, and the screenshot shows the sandbox clone's

directory listing. In one of the five runs, however, the event log has no file-change event even though the diff and screenshot both show the file, the same gap seen in 8 of the 40 benchmark runs.

7.8 Overall Discussion and Limitations

The results show that separating the execution of an agent from trust in its output costs about 1.2 seconds per run, or about 2.0 seconds with network allowlisting. In exchange, every evaluated run left the developer's checkout untouched, kept real secrets out of the agent's environment, recorded an event log, captured the change on a branch the agent could not falsify and held it behind an approval state machine.

Table 7.7: Residual risks and limitations

Limitation	Consequence	Planned improvement
Shared-kernel container isolation; default sandbox runs as root	An exploit could reach the host	Default hardening, gVisor or microVMs
Egress unrestricted unless enabled	Repository content could be exfiltrated	Default-deny egress
Host-based allowlist	Uploads to an allowlisted service are possible	Path-aware filtering
Status PATCH can set approved; no multi-user model	A token holder can skip review	Gated transitions, reviewer roles
Polling file watcher misses some writes	Timeline may omit a change	Diff cross-check, event-driven watcher
Diff tab shows counts, not patch text	Line-level review needs diff.json	Inline patch rendering

The evaluation itself is limited: all figures come from the mock agent on one machine, so they describe the pipeline's fixed overhead rather than the experience of a real agent on a large repository, and the safety results do not measure resistance to an adaptive adversary. These gaps are addressed in the Semester VIII plan.

7.9 Chapter Summary

The default pipeline completed in 1.403 s on average, adding 1.166 s to direct execution, with trusted capture the largest stage. Isolation, change capture and cleanup succeeded in every evaluated run, all injected unsafe behaviours were blocked or detected, and every value in six real sandbox credential files was a dummy. The file watcher recorded the agent's change in 80% of runs, confirming the trusted diff as the authoritative record.

Chapter 8

Conclusion

VaultShield separates running an AI coding agent from trusting what it produces. The agent receives a disposable Docker container, a fresh clone and only dummy credentials; its output, file changes and network decisions are recorded in an event log; its change is captured as one commit on a dedicated branch by a separate hardened container; and the change can become a pull or merge request only after a human approves it in the review dashboard. The system was built as a Python CLI, a Django REST Framework backend on MongoDB with Celery workers, and a React dashboard, totalling about 18.1 thousand logical lines of product code. Table 8.1 assesses the outcome against each objective.

Table 8.1: Achievement of objectives

Objective	Evidence	Outcome
1. Universal agent execution platform	Mock, codex, kiro and antigravity agents run behind one adapter interface	Achieved
2. Agentic AI Orchestrator	Not implemented in Semester VII; planned for Semester VIII	Not achieved
3. Secure sandboxed execution	Host checkout and .env unchanged in 40 of 40 runs; dummy credentials and one-shot clone token	Achieved
4. Monitoring and security analysis	Event log and diff for every run; violations recorded; Celery security scan	Achieved, with a file-watcher gap
5. Governance and human approval	Dashboard review and decision controls; submit refuses non-approved runs	Achieved

Objectives 1, 3, 4 and 5 were achieved. Objective 2, the Agentic AI Orchestrator, was not achieved: it was not implemented in Semester VII and is planned for Semester VIII. Two further gaps remain: in the single-developer MVP a token holder can bypass the dashboard through an unrestricted status update, and the polling file watcher missed some writes in very short runs, although the trusted diff captured every change. The project contributes a working implementation of the governance practices recommended for agentic systems [9], [10], engineering techniques such as shape-preserving credential substitution, a one-shot clone credential and trusted capture, and evidence that this protection costs about 1.2 seconds per run while agent-level deny rules alone do not protect secrets.

Chapter 9

Future Scope

The limitations identified in this report define the following future work.

- **Implement the Agentic AI Orchestrator:** build the module that scans the project, asks clarifying questions before execution and passes an enriched prompt to the agent, then measure whether it improves agent accuracy and reduces hallucinations, completing Objective 2.
- **Stronger Isolation:** harden the sandbox by default with a non-root user, dropped capabilities, no-new-privileges and resource limits, and support gVisor or Firecracker microVM runtimes [12], [13].
- **Policy Engine:** express rules such as required approvals for infrastructure files or approved dependency lists declaratively, and make egress default-deny with path-aware filtering.
- **Separation of Duties:** add reviewer accounts, role-based access control, decision attribution and multiple approvals, and gate every status transition through the approval endpoint.
- **Cost and Resource Controls:** record per-run token usage and enforce per-project budgets and sandbox time limits.
- **Multi-Repository and Parallel Runs:** capture one branch per repository in a single run, and let several agents attempt the same task for the reviewer to compare.
- **IDE Integration and Live Review:** start runs from the editor, render patch text in the dashboard, and stream events live so that a reviewer can interrupt a run [9].
- **Evidence Quality:** cross-check the event log against the diff, use an event-driven file watcher, and scan screenshots for credentials.
- **Broader Evaluation:** measure real agents on representative repositories, including a subset of SWE-bench [2], and run a usability study with reviewers.

Bibliography

[1] Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021.

[2] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," International Conference on Learning Representations, 2024.

[3] Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O., "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," Advances in Neural Information Processing Systems, 2024.

[4] Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., et al., "OpenHands: An Open Platform for AI Software Developers as Generalist Agents," International Conference on Learning Representations, 2025.

[5] Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., and Hashimoto, T., "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," International Conference on Learning Representations, 2024.

[6] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., and Tramèr, F., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, 2024.

[7] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M., "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," ACM Workshop on Artificial Intelligence and Security, 2023.

[8] Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., and Karri, R., "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," IEEE Symposium on Security and Privacy, 2022.

[9] Shavit, Y., Agarwal, S., Brundage, M., et al., "Practices for Governing Agentic AI Systems," OpenAI White Paper, 2023.

[10] Chan, A., Ezell, C., Kaufmann, M., Wei, K., Hammond, L., Bradley, H., et al., "Visibility into AI Agents," ACM Conference on Fairness, Accountability, and Transparency, 2024.

[11] Combe, T., Martin, A., and Di Pietro, R., "To Docker or Not to Docker: A Security Perspective," IEEE Cloud Computing, 2016.

[12] Agache, A., Brooker, M., Florescu, A., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., and Popa, D.-M., "Firecracker: Lightweight Virtualization for Serverless Applications," USENIX Symposium on Networked Systems Design and Implementation, 2020.

[13] Young, E. G., Zhu, P., Caraza-Harter, T., Arpaci-Dusseau, A. C., and Arpaci-Dusseau, R. H., "The True Cost of Containing: A gVisor Case Study," USENIX Workshop on Hot Topics in Cloud Computing, 2019.

[14] Bacchelli, A., and Bird, C., "Expectations, Outcomes, and Challenges of Modern Code Review," International Conference on Software Engineering, 2013.

[15] Boehm, B. W., "Software Engineering Economics," Prentice-Hall, 1981.